



RV College of Engineering®

Mysore Road, RV Vidyaniketan Post, Bengaluru – 560059, Karnataka, India

Go, change the world

Real-Time Human Recognition & Fall Detection Using mmWave Radar

An Interdisciplinary Project Report (XX367P)

Submitted by,

Nikhil A E

1RV23EC089

Poornachandra B R

1RV23EC095

Spoorthy S

1RV23CS305

Chirag S Shetty

1RV23CS077

Under the guidance of

Dr. Rajani Katiyar

Assistant Professor

Dept. of ECE

RV College of Engineering®

In partial fulfillment of the requirements for the degree of

Bachelor of Engineering in respective departments

2025-26



RV College of Engineering®, Bengaluru
(Autonomous institution affiliated to VTU, Belagavi)



CERTIFICATE

Certified that the interdisciplinary project (XX367P) work titled ***Real-Time Human Recognition & Fall Detection Using mmWave Radar*** is carried out by **Nikhil A E (1RV23EC089)**, **Poornachandra B R (1RV23EC095)**, **Spoorthy S (1RV23CS305)** and **Chirag S Shetty (1RV23CS077)** who are bonafide students of RV College of Engineering, Bengaluru, in partial fulfillment of the requirements for the degree of **Bachelor of Engineering** in respective departments during the year 2025-26. It is certified that all corrections/suggestions indicated for the Internal Assessment have been incorporated in the interdisciplinary project report deposited in the departmental library. The interdisciplinary project report has been approved as it satisfies the academic requirements in respect of interdisciplinary project work prescribed by the institution for the said degree.

Dr. Rajani Katiyar
Guide

Dr. Ravish Aradhya
Head of the Department

Dr. M.V. Renukadevi
Dean Academics

Dr. K. N. Subramanya
Principal

External Viva

Name of Examiners

Signature with Date

1.

2.

DECLARATION

We, **Nikhil A E, Poornachandra B R, Spoorthy S, Chirag S Shetty** students of sixth semester B.E., RV College of Engineering, Bengaluru, hereby declare that the interdisciplinary project titled '**Real-Time Human Recognition & Fall Detection Using mmWave Radar**' has been carried out by us and submitted in partial fulfilment for the award of degree of **Bachelor of Engineering** in respective departments during the year 2025-26.

Further we declare that the content of the dissertation has not been submitted previously by anybody for the award of any degree or diploma to any other university.

We also declare that any Intellectual Property Rights generated out of this project carried out at RVCE will be the property of RV College of Engineering, Bengaluru and We will be one of the authors of the same.

Place: Bengaluru

Date:

Name

Signature

1. Nikhil A E(1RV23EC089)
2. Poornachandra B R(1RV23EC095)
3. Spoorthy S(1RV23CS305)
4. Chirag S Shetty(1RV23CS077)

ACKNOWLEDGEMENTS

We are indebted to our guide, **Dr. Rajani Katiyar**, Assistant Professor, Department of ECE, RVCE for the wholehearted support, suggestions and invaluable advice throughout our project work and also helped in the preparation of this thesis.

We also express our gratitude to our panel member **Dr. B S Kariyappa**, Professor, Department of ECE, RVCE for their valuable comments and suggestions during the phase evaluations.

Our sincere thanks to the project coordinator **Prof. Sujata Priyambada Mishra**, Professor, Department of ECE for timely instructions and support in coordinating the project.

Our gratitude to **Prof. Narashimaraja P**, Department of ECE, RVCE for the organized latex template which made report writing easy and interesting.

Our sincere thanks to **Dr. Ravish Aradhya**, Professor and Head, Department of Electronics and Communication Engineering, RVCE for the support and encouragement.

We are deeply grateful to **Dr. M.V. Renukadevi**, Professor and Dean Academics, RVCE, for the continuous support in the successful execution of this Interdisciplinary project.

We express sincere gratitude to our beloved Professor and Vice Principal, **Dr. Geetha K S**, RVCE and Principal, **Dr. K. N. Subramanya**, RVCE for the appreciation towards this project work.

Lastly, we take this opportunity to thank our family members and friends who provided all the backup support throughout the project work.

ABSTRACT

This project presents a comprehensive, real-time fall detection system leveraging dual Texas Instruments IWR6843AOP FMCW mmWave radar sensors. Designed to overcome the occlusion and privacy limitations of traditional cameras, the system utilizes a robust track-based auto-calibration pipeline to fuse data from two independent radars—placed anywhere in a room—into a single unified 3D point cloud, without requiring manual measurement or hardware synchronization.

The system architecture features an optimized edge-computing pipeline that performs nearest-frame timestamp synchronization, calculates rigid spatial transforms using SVD and RANSAC, and merges the point clouds. To ensure high-quality data, redundant detections are merged via voxel downsampling (0.1 m grid), and multipath ghost points are eliminated using Statistical Outlier Removal (SOR). This self-healing pipeline automatically detects sensor drift and recalibrates in real time.

The unified point cloud is subsequently fed into a hybrid Transformer-CNN-LSTM deep learning architecture. This model extracts global temporal dependencies, local kinematic patterns, and sequential motion memory to classify sequences precisely as "Fall" or "No-Fall" events. Real-time predictions are pushed to a Supabase PostgreSQL database and visualized on a React dashboard via WebSocket channels. The end-to-end fusion and classification pipeline demonstrates sub-second latency, providing a highly scalable, privacy-first solution for continuous ambient safety monitoring.

CONTENTS

Abstract	i
List of Figures	v
List of Tables	vi
1 Introduction	1
1.1 Introduction	2
1.2 Literature Review	3
1.3 Motivation	4
1.4 Problem Statement	5
1.5 Objectives	6
1.6 Brief Methodology of the Project	6
1.7 Assumptions Made / Constraints of the Project	7
1.8 Organization of the Report	7
2 Hardware and Data Fundamentals	9
2.1 Dual-Sensor Hardware Configuration	10
2.1.1 AWR6843AOP	10
2.1.2 IWR6843AOP	10
2.1.3 Shared Hardware Parameters	11
2.2 Host Edge Platform: Raspberry Pi	11
2.3 Firmware: 3D People Tracking	12
2.4 Understanding Sensor Output: The JSON Frame	12
2.4.1 Point Cloud vs. Targets: Design Choice	12
3 System Architecture Overview	14
3.1 Three-Tier Architecture	15
3.2 Inter-Tier Communication Protocols	15
3.3 Containerisation and Deployment	17
3.4 Amazon Web Services (AWS) Infrastructure	17
3.4.1 AWS Overview	17

3.4.2	AWS EC2 — Elastic Compute Cloud	18
3.4.3	AWS DynamoDB — NoSQL Event Store	19
3.4.4	AWS S3 — Static Website Hosting for the React Dashboard	20
3.4.5	Docker and Nginx on EC2	21
3.4.6	Summary of AWS Services Used	22
4	Dual-Radar Fusion Pipeline	23
4.1	Stage 1: Frame Synchronization	24
4.2	Stage 2: Auto-Calibration via SVD + RANSAC	24
4.2.1	Step 1: Collecting Correspondences	24
4.2.2	Steps 2–5: SVD Computation	24
4.2.3	Step 6: RANSAC Outlier Filtering	26
4.2.4	Drift Detection and Auto-Recalibration	27
4.3	Stage 3: Point Cloud Fusion and Cleanup	27
4.3.1	Rigid Transform Application	27
4.3.2	Concatenation	27
4.3.3	Voxel Downsampling	28
4.3.4	Statistical Outlier Removal (SOR)	28
4.3.5	Pipeline Summary	29
5	Feature Extraction and Fall Detection	30
5.1	Feature Extraction	31
5.2	Cloud Inference Infrastructure	32
5.3	Rule-Based Fall Detection: <i>StreamingFallDetector</i>	32
5.3.1	Design Rationale	32
5.3.2	Three-Test Majority Vote	33
5.3.3	Persistence and Real-Time Broadcast	33
5.4	Heterogeneous Fusion Results	35
5.5	Real-Time Dashboard	36
6	Deep Learning Architecture: Transformer-CNN-LSTM	37
6.1	Rationale for Hybrid Architecture	38
6.2	Architecture Layout	38
6.3	Training Configuration	38

6.4	Integration Path	39
7	Results & Discussions	40
7.1	Algorithm Comparison: SVD + RANSAC vs. Alternatives	41
7.2	Fall Detector Comparison	41
7.3	Cloud Infrastructure Design Decisions	42
7.4	End-to-End Latency	42
7.5	AWS Deployment: Screenshots and Validation	42
7.5.1	Live Dashboard — Fall Alert State (AWS EC2 + S3)	43
7.5.2	Live Dashboard — Inference History and Fall Event	44
7.5.3	Feature Count Validation	45
7.5.4	Summary of Results	45
8	Conclusion and Future Scope	47
8.1	Conclusion	48
8.2	Future Scope	48
8.3	Learning Outcomes of the Project	49
A	Supplementary Technical Details	50
A.1	Raspberry Pi UART Data Parser Architecture	51
	Bibliography	52

LIST OF FIGURES

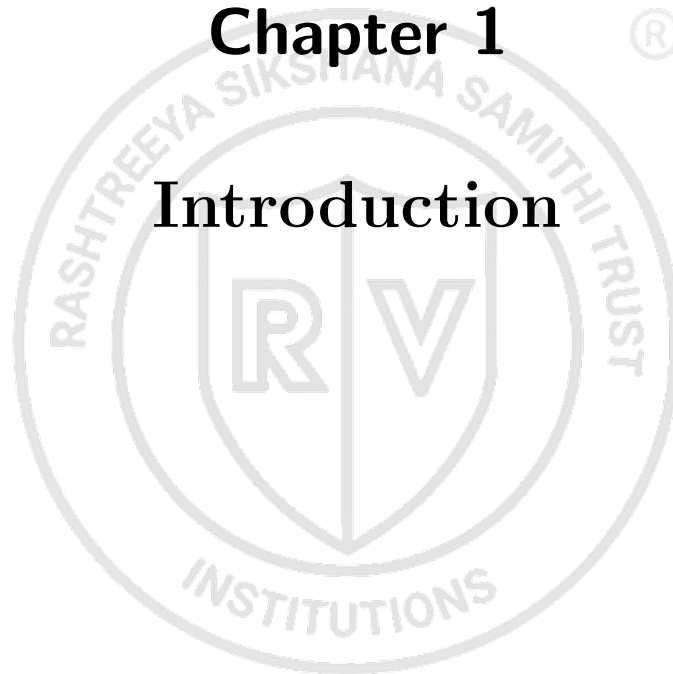
3.1	End-to-end system architecture	16
4.1	End-to-end dual-radar fusion pipeline	25
4.2	RANSAC-based Auto-Calibration Convergence	26
4.3	Dual-Radar Point Cloud Fusion — Initial Merge	28
4.4	Dual-Radar Point Cloud — Final Fused Output	28
5.1	<i>StreamingFallDetector</i> per-frame decision flow	34
5.2	Single-Person Point Cloud Validation: IWR6843 vs. AWR1843 vs. Fused Output	35
5.3	Real-Time Activity Dashboard	36
7.1	AWS S3-Hosted Dashboard — Fall Alert State (66.7% confidence, 200 events logged)	43
7.2	AWS Dashboard — Inference History with Expanded FALL Event (Frame #140, 90% confidence)	44

LIST OF TABLES

2.1	Key Radar Hardware Parameters	11
2.2	JSON Frame Fields from 3D People Tracking Firmware	12
3.1	AWS Services Used in the Radar Fall Detection System	22
4.1	Summary of the Dual-Radar Fusion Pipeline	29
5.1	20-Dimensional Feature Vector Extracted per Radar Frame	31
5.2	Key Fields of the <code>/frames/batch</code> API Endpoint	32
5.3	<i>StreamingFallDetector</i> Three-Test Majority Vote Criteria	33
7.1	Comparison of Dual-Radar Calibration Approaches	41
7.2	Comparison: Rule-Based vs. Transformer-CNN-LSTM Fall Detector . . .	42
7.3	End-to-End Latency Budget	42
7.4	Summary of System Validation Results	46

Chapter 1

Introduction



The following chapter presents the background, motivation, and scope of this interdisciplinary project. It begins by contextualizing the healthcare problem of elderly falls and the shortcomings of existing detection technologies, then formalizes the problem statement, states the project objectives, and describes the organization of the remainder of this report.

1.1 Introduction

Real-time automated fall detection is a critical requirement in modern healthcare, geriatric care, and ambient assisted living (AAL). Falls represent the leading cause of both fatal and non-fatal injuries among elderly adults worldwide. According to the World Health Organization (WHO), approximately 684,000 fatal falls occur each year globally, making falls the second leading cause of unintentional injury death after road traffic injuries. Among adults aged 60 years or older, falls are the leading cause of injury and death from injury. In India, the rapidly ageing population creates an urgent and growing demand for scalable, affordable, and non-intrusive monitoring solutions that can detect falls in real time and notify caregivers or emergency services without delay.

The clinical and economic consequences of a fall extend well beyond the initial physical injury. A significant proportion of elderly individuals who fall but are not quickly discovered experience what is clinically termed a “long lie”—a prolonged period spent on the ground before receiving assistance, which dramatically worsens prognosis through dehydration, pressure sores, hypothermia, rhabdomyolysis, and severe psychological trauma.

Long-lie episodes of even one hour are associated with significantly increased short-term mortality. Rapid detection and emergency response can therefore significantly reduce associated mortality and morbidity, making

automated fall detection systems a meaningful and important intervention in elder care infrastructure.

This project implements a privacy-preserving fall detection system utilizing two independent 60 GHz FMCW mmWave radar sensors: a Texas Instruments AWR6843AOP (automotive-grade) and an IWR6843AOP (industrial-grade). The mmWave radar modality was chosen because it generates only abstract 3D point clouds rather than any identifiable visual imagery, making it uniquely suited to deployment in privacy-sensitive locations such as bedrooms and bathrooms. The system is designed from the ground up to be deployable in real-world domestic environments without any specialized installation

expertise, manual calibration procedures, or dedicated network infrastructure.

1.2 Literature Review

A substantial body of literature addresses the fall detection problem using diverse sensing modalities. The major approaches reported in the literature are wearable sensors, visionbased systems, pressure-sensitive floors, and ambient radio-frequency (RF) sensing, each with distinct advantages and limitations.

Wearable Sensor Approaches: Inertial measurement units (IMUs), accelerometers, and gyroscopes embedded in smartwatches or purpose-built pendants represent the most widely deployed commercial approach. Bourke and Lyons [1] demonstrated that thresholdbased classifiers on tri-axial accelerometer data can achieve high sensitivity for simulated falls in controlled conditions. However, compliance is a critical limitation: elderly individuals, particularly those with cognitive impairment, may forget or refuse to wear the device—precisely during the activities (bathing, night movement) when falls are most likely. A device worn inconsistently provides no protection during the periods of noncompliance.

Camera-Based Approaches: Vision-based systems using RGB, depth (RGB-D), or thermal cameras offer high spatial resolution and can leverage deep learning for accurate activity classification. Noury et al. [2] reviewed numerous vision-based fall detection approaches and demonstrated high accuracy in structured environments. However, deployment of cameras in bedrooms or bathrooms is unacceptable to most individuals due to severe privacy concerns. Camera-based algorithms also fail under poor or zero lighting and are sensitive to occlusion by furniture.

Radar-Based Approaches: RF sensing using Wi-Fi, Ultra-Wideband (UWB), or mmWave radar has attracted increasing research interest. mmWave FMCW radar, operating in the 60–64 GHz band, offers unique advantages: privacy-preserving point cloud output (no identifiable visual information), operation independent of ambient lighting, and penetration through light physical obstructions. Jokanovic et al. [3] demonstrated that micro-Doppler signatures from 77 GHz FMCW radar can discriminate falls from activities of daily living using machine learning classifiers.

Multi-Sensor Fusion: Combining multiple sensors to eliminate blind spots and improve robustness has been explored in several contexts. Point cloud registration algorithms such as Iterative Closest Point (ICP) [4] and SVD-based Procrustes alignment

[5] provide principled mathematical frameworks for spatial registration. RANSAC [6] enables robust estimation in the presence of outliers, a critical property for mmWave radar which generates sparse and noisy point clouds compared to LiDAR. The application of these established algorithms to the specific challenge of dual-radar auto-calibration for fall detection is novel and constitutes the core contribution of this work.

Cloud-Edge Architectures for IoT Health Monitoring: The deployment of health monitoring systems on AWS or similar cloud platforms has been documented extensively. FastAPI [7], DynamoDB, and WebSocket-based real-time notification patterns provide well-established primitives for building low-latency health alert systems.

The review reveals a gap: existing radar-based fall detection systems either require a single radar (susceptible to blind spots) or multiple radars with manual calibration (brittle and maintenance-intensive). No published work addresses the automatic, online calibration of multiple mmWave radars using only their tracking output in a real-time streaming context. This gap motivates the auto-calibration pipeline presented in this report.

1.3 Motivation

A single radar sensor can suffer from occlusion or a limited field of view in complex room geometries with furniture, corners, or partitioned spaces. A person who falls behind a large piece of furniture or in a corner not covered by the sensor's field of view would go undetected. Utilizing a dual-radar configuration ensures comprehensive room coverage and eliminates sensor blind spots that would allow a fall to go undetected.

However, the core challenge in a dual-radar system is combining detections from two radars that report positions relative to their own distinct, independent coordinate frames into a single coherent spatial representation. Manual calibration—physically measuring sensor placement with tape measures and protractors—is error-prone and brittle: angular measurement errors of 2–3 degrees produce positional errors of approximately 35 cm at 10 m range. More critically, manual calibration breaks silently whenever a sensor is inadvertently nudged, tilted, or repositioned—undetectable without repeating the entire measurement process. In a real domestic deployment, sensors may be moved during room cleaning, furniture rearrangement, or by children, rendering the calibration invalid.

A self-calibrating pipeline that continuously monitors calibration quality and can autonomously recalibrate when sensor displacement is detected would make the system truly

deployable and maintenance-free for non-technical users. This operational resilience is the primary motivation for the auto-calibration approach developed in this project.

Furthermore, the choice of a rule-based fall detector over a machine-learning model is motivated by the absence of a labelled training dataset and the need for immediate deployability. A deterministic, physics-grounded classifier requires no training data, executes with sub-millisecond latency on any CPU, and is fully interpretable—all critical properties for a safety-critical system where false negatives (missed falls) and false positives (spurious alerts) both carry real consequences.

1.4 Problem Statement

The two radar units operate on independent internal clocks with no hardware synchronization signal between them. Each unit delivers frames to the host at slightly different absolute times. Naively combining their frames produces temporal mismatch and spatial

distortion: a person walking at 1 m/s moves approximately 4 cm per 40 ms offset—sufficient to corrupt geometric calibration and introduce ghost detections in the fused point cloud. Furthermore, the two sensors are placed at arbitrary positions and orientations in a room, so their coordinate systems are related by an unknown 6-degrees-of-freedom (6-DOF) rigid body transformation that must be determined automatically.

The problem is to design an end-to-end edge-to-cloud architecture that:

- (1) **Synchronizes** incoming frames via timestamps to eliminate temporal mismatch between the two unsynchronized sensors;
- (2) **Auto-calibrates** by mathematically deducing the rigid spatial transform between the sensors using only their observed tracking output, requiring no manual measurement;
- (3) **Fuses and cleans** the resulting point clouds to produce a high-quality unified 3D representation free of duplicates and noise;
- (4) **Autonomously detects and corrects** sensor displacement-induced calibration drift without human intervention;
- (5) **Classifies** each fused frame using a deterministic, rule-based fall detector that evaluates physical signatures of a fall event; and
- (6) **Delivers** binary fall/no-fall decisions to a remote dashboard with sub-400 ms end-to-end latency.

1.5 Objectives

The specific technical objectives of this project are:

1. Design a dual-radar synchronization module based on nearest-timestamp frame matching, capable of pairing frames from two unsynchronized USB serial streams at approximately 10 FPS each.
2. Implement a track-based auto-calibration pipeline using SVD and RANSAC to dynamically compute the rigid transform (R, t) between sensors from a brief calibration walk of 20–30 s, without any external measurement tools.
3. Fuse and clean the unified point cloud using voxel downsampling at 0.1 m resolution and Statistical Outlier Removal with $K = 5$ nearest neighbours to produce a denoised, deduplicated representation.
4. Deploy a stateful, deterministic rule-based fall detector (*StreamingFallDetector*) on AWS EC2 that evaluates three complementary physical signatures of a fall event per radar frame using a majority vote mechanism.
5. Persist confirmed fall events in AWS DynamoDB and broadcast all inference results via WebSocket to a React dashboard hosted on AWS S3, achieving sub-400 ms end-to-end latency.
6. Design and document a Transformer-CNN-LSTM deep learning architecture as a future upgrade path for enhanced classification accuracy, compatible with the same 20-dimensional feature vector produced by the edge pipeline.
7. Evaluate the complete system with respect to latency, calibration accuracy, fusion quality, and detection logic correctness.

1.6 Brief Methodology of the Project

The project follows a three-tier architecture: an edge tier on the Raspberry Pi for realtime sensor reading, synchronization, auto-calibration, fusion, and feature extraction; a cloud inference tier on AWS EC2 for rule-based fall classification and persistence; and a presentation tier on AWS S3 for the real-time React dashboard.

The auto-calibration methodology involves collecting point-cloud correspondences from a brief calibration walk, computing the optimal rotation matrix R via Singular Value Decomposition of the cross-covariance matrix of correspondence pairs, and computing the translation vector t analytically. RANSAC is applied to reject outlier correspondences before the final SVD pass. The resulting (R, t) is applied per-frame to transform all

Radar B points into Radar A's coordinate frame.

Fall classification uses three deterministic threshold tests on the 20-dimensional feature vector extracted from each fused frame: a centroid height drop test, a body flatness test, and a point count test. A majority vote of at least 2 of 3 tests triggers a FALL classification.

1.7 Assumptions Made / Constraints of the Project

The following assumptions and constraints apply to this project:

- Both radars are rigidly mounted during operation; the rigid body assumption underlying the SVD calibration holds as long as sensors are not moved.
- The calibration environment contains a single walking person during the calibration period to avoid correspondence ambiguity.
- The environment is assumed to be quasi-static (no large moving objects other than the monitored person) during normal operation.
- The edge device (Raspberry Pi) maintains a continuous network connection to AWS EC2 for batch uploads; occasional brief disconnections can be tolerated by retrying the HTTP POST.
- The rule-based fall detector is calibrated for a single person in the monitored space; multi-person scenarios are flagged as a limitation for future work.
- The system is designed for indoor use in domestic environments; outdoor or highly metallic environments may alter the radar's multipath characteristics.

1.8 Organization of the Report

The remainder of this report is organized as follows:

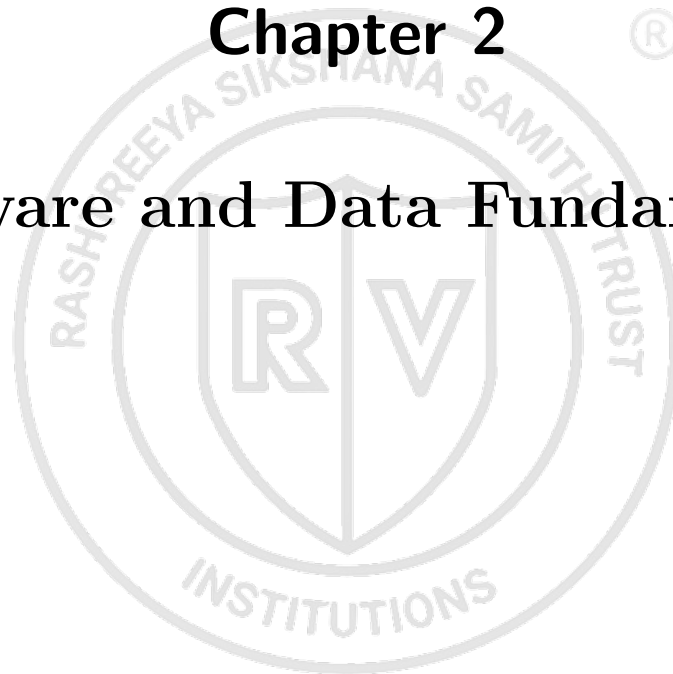
- **Chapter 2:** Hardware and Data Fundamentals describes the dual-sensor hardware configuration, the 3D People Tracking firmware, and the structure of the JSON frame output from each sensor.
- **Chapter 3:** System Architecture Overview presents the complete three-tier end-to-end pipeline from sensor capture to dashboard visualization, and describes inter-tier communication protocols and deployment infrastructure.
- **Chapter 4:** Dual-Radar Fusion Pipeline details the complete real-time synchronization, auto-calibration (SVD+RANSAC), drift detection, point cloud fusion, voxel

downsampling, and Statistical Outlier Removal stages.

- **Chapter 5:** Feature Extraction and Fall Detection covers the 20-dimensional feature vector definition, the cloud inference infrastructure, the rule-based *Streaming-FallDetector* and its three-test majority vote, and the real-time React dashboard.
- **Chapter 6:** Deep Learning Architecture documents the Transformer-CNN-LSTM upgrade path designed for future deployment when labelled data becomes available.
- **Chapter 7:** System Evaluation and Results presents algorithm comparisons, infrastructure design decisions, and the end-to-end latency budget.
- **Chapter 8:** Conclusion and Future Scope summarizes contributions, discusses limitations, and proposes directions for future work.
- **Appendix A:** Raspberry Pi UART Parser Architecture describes the asynchronous two-thread parser architecture for TLV binary frame decoding from USB serial.

Chapter 2

Hardware and Data Fundamentals



This chapter describes the hardware platform upon which the system is built. It covers the dual-sensor radar configuration, the on-chip firmware, and the structure of the data output from each sensor. Understanding the hardware capabilities and output format is essential context for the fusion and detection pipeline described in subsequent chapters.

2.1 Dual-Sensor Hardware Configuration

The system employs a heterogeneous radar pair from Texas Instruments: one AWR6843AOP (automotive-grade) and one IWR6843AOP (industrial-grade). Both devices are functionally equivalent for this application and share the same physical dimensions, antenna layout, and supported firmware. The use of one unit from each product family reflects hardware availability during development; the fusion pipeline treats both sensors uniformly, with no special handling for minor differences between the two product lines.

The choice of AWR6843AOP and IWR6843AOP was driven by several considerations: (1) the Antenna-on-Package (AOP) design reduces form factor and eliminates external RF connectors, simplifying physical deployment; (2) both devices integrate all signal processing on-chip, reducing the computational demand on the host edge device; (3) the 60–64 GHz mmWave band provides the sub-centimetre range resolution and Doppler velocity sensitivity required to discriminate between standing, walking, sitting, and lying postures; and (4) the Texas Instruments ecosystem provides a mature firmware ecosystem and open-source host software libraries that accelerate development.

2.1.1 AWR6843AOP

The AWR6843AOP is a single-chip 60–64 GHz FMCW radar designed for automotive-grade applications. It features an Antenna-on-Package (AOP) design producing an extremely compact footprint (10.4×10.4 mm). The device hosts an on-chip ARM CortexR4F and a dedicated hardware accelerator (HWA) for FFT-based signal processing, along with a C674x DSP for CFAR detection, DBSCAN clustering, and Kalman filter-based person tracking—all executing internally before any data reaches the host computer.

2.1.2 IWR6843AOP

The IWR6843AOP is the industrial-grade variant sharing the same silicon architecture. It operates over the same 60–64 GHz frequency band with the same MIMO antenna array

(3 TX, 4 RX). The primary differences are extended operating temperature ratings and industrial qualification levels; for indoor fall detection at standard room temperature, both sensors are operationally identical.

2.1.3 Shared Hardware Parameters

Key parameters common to both sensors are summarized in Table 2.1.

Table 2.1: Key Radar Hardware Parameters

Parameter	Value	Significance
Frequency Band	60–64 GHz	mmWave; privacy-preserving, works in darkness
Bandwidth	3.99 GHz	Range resolution ≈ 4 cm theoretical
TX Antennas	3	Enables 3D angle-of-arrival estimation
RX Antennas	4	12 virtual MIMO elements for spatial resolution
On-chip DSP	ARM R4F + C674x	CFAR, DBSCAN, EKF tracking on-chip
USB Baud Rate	921,600 baud	High-speed UART over USB CDC
Frame Rate	≈ 10 FPS	100 ms frame period; adequate for fall detection
Operational Range	0.5–20 m	Covers typical room or corridor
Point Cloud Output	x, y, z, v, snr	3D position, radial velocity, signal quality

2.2 Host Edge Platform: Raspberry Pi

The host edge computing platform is a Raspberry Pi single-board computer. Both radar sensors connect to the Raspberry Pi via USB CDC serial at 921,600 baud. The Raspberry Pi handles real-time data ingestion, frame synchronization, auto-calibration, point cloud fusion, feature extraction, and file batching. The core pipeline is implemented in Python using NumPy for vectorized point cloud arithmetic and SciPy for SVD computation. No GPU is required on the edge device; all computations complete in under 10 ms per frame on the ARM CPU.

2.3 Firmware: 3D People Tracking

Both sensors are flashed with Texas Instruments’ “3D People Tracking” firmware (lab0012). This firmware offloads computational burden to the sensor’s on-chip DSP, leaving the host free for multi-sensor fusion. Internally, the firmware executes:

1. **CFAR Detection:** Constant False Alarm Rate thresholding identifies radar reflections above the noise floor, converting the range-Doppler map into a sparse set of detected reflection candidates.
2. **DBSCAN Clustering:** Groups nearby reflections into coherent clusters representing distinct physical objects.
3. **Extended Kalman Filter (EKF) Tracking:** Associates clusters across frames, maintaining stable track IDs (‘tid’) and providing smooth centroid estimates.

2.4 Understanding Sensor Output: The JSON Frame

Raw binary telemetry is transmitted at 921,600 baud over USB using TLV (TypeLength-Value) binary encoding. The host parser synchronizes via an 8-byte magic word (‘0x0201040306050807’), extracts a 32-byte header, and decodes three TLV structures: ‘TLVPOINTCLOUD’ (x, y, z , Doppler velocity, SNR), ‘TLVTRACK3D’ (centroid and bounding box per tracked person), and ‘TLVTRACKINDEX’ (mapping from reflection points to their associated track). These are assembled into a structured JSON frame whose key fields are described in Table 2.2.

Table 2.2: JSON Frame Fields from 3D People Tracking Firmware

Field	Type	Description
‘timestamp’	Float	System time of frame capture (s)
‘pointCloud’	Array	Variable-length list of detected points (x, y, z, v, snr, track_id)
‘targets’	Array	Variable-length list of tracked people (tid, x, y, z, velocity)

2.4.1 Point Cloud vs. Targets: Design Choice

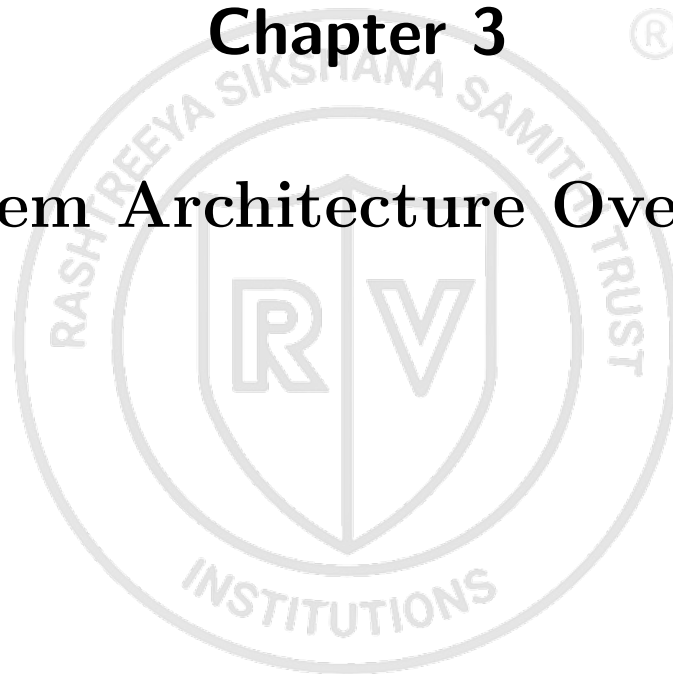
The ‘targets’ array provides a smooth, noise-reduced centroid per person—ideal for geometric calibration. However, it discards all body-shape information. A fall is characterized by a rapid spatial transition from a tall, narrow point cloud (standing) to a wide, low, flat profile (lying)—shape information that exists only in the raw ‘pointCloud’

array. Accordingly, the calibration module uses ‘targets’ centroids for robust spatial alignment, while the fall detector analyzes the full fused ‘pointCloud’ to capture body-shape signatures.



Chapter 3

System Architecture Overview



This chapter presents the overall system architecture organised into three functional tiers. The three-tier decomposition enables independent development, testing, and scaling of each component. Figure 3.1 presents the complete end-to-end pipeline from sensor capture to dashboard visualisation.

3.1 Three-Tier Architecture

1. Edge Tier (Raspberry Pi): Two radar sensors deliver raw TLV frames over USB. A dedicated reader thread per sensor timestamps and buffers each frame. A pairing thread synchronises frames across both sensors, drives the auto-calibration and fusion pipeline, extracts a 20-dimensional feature vector per frame, and batches ~ 36 frames into a single JSON file written to a local watch directory approximately once per second.

2. Cloud Inference Tier (AWS EC2): A file watcher detects each new batch file via Linux inotify and POSTs the full batch to a FastAPI service running inside a Docker container on AWS EC2 (region `ap-south-1`, Mumbai). The API applies a stateful, rule-based fall detector per device, persists confirmed fall events to AWS DynamoDB with a 24-hour TTL, and broadcasts every inference result to connected dashboard clients via WebSocket.

3. Presentation Tier (AWS S3 + Browser): A React single-page application hosted on AWS S3 static website hosting connects to the WebSocket endpoint on page load, maintains a rolling buffer of 200 recent events, and renders live activity metrics and fall alerts.

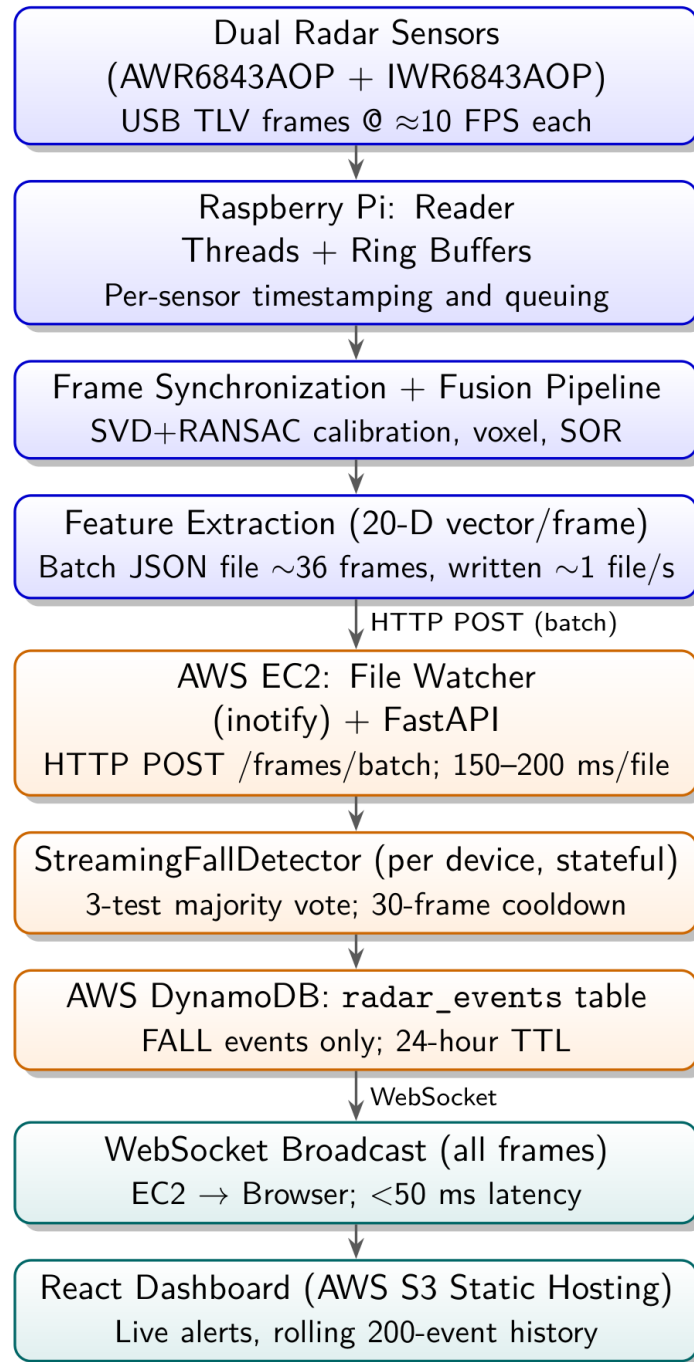
3.2 Inter-Tier Communication Protocols

Between the edge tier and cloud tier, communication occurs via HTTP POST requests carrying JSON-encoded batch files. The use of stateless HTTP rather than a persistent TCP connection simplifies retry logic: if a POST fails due to transient network interruption, the edge device can re-attempt without any server-side state synchronisation.

Between the cloud tier and the presentation tier, communication occurs via a persistent WebSocket connection, providing server-push capability for immediate alert delivery.

WebSocket is chosen over periodic HTTP polling because it eliminates polling latency and allows the server to push each inference result to the browser the instant it is produced, essential for achieving sub-400 ms end-to-end latency.

lerts.



system architecture: edge radar capture and fusion

Figure 3.1: End-to-end system architecture

3.3 Containerisation and Deployment

The FastAPI application on EC2 is packaged inside a Docker container, providing environment reproducibility and simplifying deployment. Nginx acts as a reverse proxy in front of Uvicorn, handling TLS termination and WebSocket upgrade negotiation. The Docker-based deployment ensures the cloud inference stack can be reproduced identically on any EC2 instance type with minimal configuration changes, and simplifies blue-green deployment for updates.

3.4 Amazon Web Services (AWS) Infrastructure

This project relies on three core AWS services — EC2, DynamoDB, and S3 — each selected for its unique properties in the context of real-time, serverless-friendly, event-driven radar inference.

3.4.1 AWS Overview

Amazon Web Services (AWS) is the world's most comprehensive and broadly adopted cloud platform, offering over 200 fully featured services from data centres globally. AWS enables organisations to lower costs, become more agile, and innovate faster by removing the need to own and maintain physical infrastructure. The key characteristics of AWS relevant to this project are:

- **Global Regions and Availability Zones:** AWS infrastructure is divided into geographic Regions, each containing multiple isolated Availability Zones (AZs). This project uses the `ap-south-1` (Mumbai) Region, chosen for minimal round-trip latency to the edge device located in India.
- **Pay-as-you-go Pricing:** Resources are billed only for actual usage, making prototyping cost-efficient.
- **Managed Services:** AWS abstracts operating-system-level administration for services like DynamoDB and S3, eliminating patching, scaling, and backup overhead.
- **Security and IAM:** AWS Identity and Access Management (IAM) provides fine-grained, role-based access control. The EC2 instance is assigned an IAM role granting least-privilege access to only the `radar_events` DynamoDB table, following the principle of least privilege.

3.4.2 AWS EC2 — Elastic Compute Cloud

Service Description: Amazon Elastic Compute Cloud (EC2) provides resizable virtual machine instances in the cloud. An EC2 instance is a virtual server with a configurable combination of CPU, memory, storage, and networking capacity. Instances are launched from *Amazon Machine Images* (AMIs), which bundle the operating system and pre-installed software.

Role in this Project: EC2 hosts the cloud inference tier. Specifically:

- A **t3.micro** instance (2 vCPU, 1 GB RAM) running Amazon Linux 2 is deployed in the **ap-south-1** (Mumbai) Region. This instance type is sufficient because the rule-based *StreamingFallDetector* executes in under 1 ms per frame, leaving ample headroom.
- The FastAPI application (**radar-api-main.py**) is containerised using **Docker** and served by **Uvicorn** (ASGI server) behind **Nginx** (reverse proxy). Nginx handles port 80 traffic, WebSocket upgrade headers, and distributes requests to Uvicorn.
- An **Elastic IP address** is assigned to the instance, providing a static public IP that survives instance stop/start cycles. The IP **43.205.167.81** is the publicly reachable endpoint used by the Raspberry Pi and the React dashboard.
- **Security Groups** (virtual firewalls) restrict inbound traffic to only port 80 (HTTP/WebSocket) and port 22 (SSH for administration), rejecting all other traffic.

Key EC2 Features Utilised:

- **Elastic IP:** A static public IPv4 address that does not change across reboots, ensuring the Raspberry Pi and browser always reach the same endpoint without DNS configuration.
- **Security Groups:** Stateful firewall rules at the instance level. Inbound rules allow TCP/80 from 0.0.0.0/0 (any IP) for the API and WebSocket, and TCP/22 from the developer's IP for SSH.
- **IAM Instance Profile:** Grants the EC2 process permission to call `dynamodb:PutItem` and `dynamodb:Query` on the `radar_events` table without embedding credentials in the application code.

- **User Data Script:** A startup script installs Docker, pulls the application image, and launches the container on every boot, enabling fully automated re-provisioning.
- **CloudWatch Logs:** Uvicorn application logs are streamed to AWS CloudWatch, enabling remote log inspection without SSH access.

3.4.3 AWS DynamoDB — NoSQL Event Store

Service Description: Amazon DynamoDB is a fully managed, serverless, key-value and document NoSQL database that delivers single-digit millisecond performance at any scale. It requires no capacity planning for reads/writes below the provisioned throughput threshold and provides built-in high availability by replicating data across three Availability Zones within a Region.

Role in this Project: DynamoDB serves as the persistent event log for confirmed fall events:

- **Table name:** `radar_events`
- **Partition key:** `device_id` (String) — uniquely identifies each radar node (e.g. `rpi-1`).
- **Sort key:** `ts` (String, ISO 8601 UTC) — enables time-ordered queries within a device's event history.
- **Global Secondary Index:** A pk attribute (always set to "all") allows scanning across all devices when a unified event feed is required.
- **TTL (Time to Live):** Each item carries an `expire_at` attribute set to 24 hours after insertion. DynamoDB's TTL feature automatically deletes expired items, preventing unbounded table growth and eliminating the need for a scheduled purge job.
- **Write path:** Only FALL-classified frames trigger a `PutItem` call, keeping write throughput low (typically <1 write per minute per device under normal conditions).
- **Read path:** The `GET /history` endpoint issues a `Query` using the `device_id` partition key with `ScanIndexForward=False` to retrieve the most recent events in descending timestamp order.

Rationale for DynamoDB over alternatives:

- **Schema-free:** No migration required when new debug fields (e.g. `z_drop`, `drop_streak`) are added to the fall event record.
- **Built-in TTL:** Automatic data expiry without a cron job or Lambda function.
- **Horizontal scalability:** Write throughput scales with additional radar nodes without database reconfiguration.
- **Single-digit millisecond writes:** The `PutItem` call completes in approximately 30 ms (Table 7.3) — entirely within the 400 ms system latency budget.

3.4.4 AWS S3 — Static Website Hosting for the React Dashboard

Service Description: Amazon Simple Storage Service (S3) is an object storage service offering industry-leading scalability, data availability (99.999999999% durability), security, and performance. S3 stores objects (files) inside *buckets*. One of its built-in features is **Static Website Hosting**, which serves the contents of a bucket as a public website without requiring a web server.

Role in this Project: The React single-page application (SPA) is compiled into a set of static HTML, CSS, and JavaScript files and uploaded to an S3 bucket configured for static website hosting. Key configuration details:

- **Bucket name:** `radar-dashboard` (or similar, in `ap-south-1`).
- **Static Website Hosting:** Enabled with `index.html` as both the index document and the error document (required for client-side routing).
- **Bucket Policy:** A public-read bucket policy grants `s3:GetObject` to any principal (`"Principal": "*"`), making all files publicly accessible.
- **CORS Configuration:** A Cross-Origin Resource Sharing (CORS) rule allows the browser served from the S3 URL to open a WebSocket connection to the EC2 instance at a different origin.

Dashboard Behaviour: On page load, the React application:

1. Issues a `GET /history?device_id=rpi-1&limit=60` REST call to the EC2 API to pre-populate the inference history panel with recent fall events stored in DynamoDB.
2. Opens a persistent WebSocket connection to `ws://<EC2-IP>/ws`.
3. Renders incoming JSON messages as live inference cards in real time.
4. On a "FALL" event, switches the UI to an alert state with a prominent visual indicator.

Rationale for S3 Static Hosting over alternatives (e.g. EC2-served React):

- **Zero server maintenance:** No web server process to manage, patch, or monitor.
- **99.99% availability SLA:** S3 static hosting provides higher availability than a single EC2 instance.
- **Global edge delivery:** S3 can be paired with CloudFront (AWS CDN) for low-latency global delivery with a single configuration change.
- **Cost:** Static file hosting costs only storage (~\$0.023/GB-month) and data transfer, an order of magnitude cheaper than running a web server on EC2 24/7.

3.4.5 Docker and Nginx on EC2

Docker: The FastAPI application and all its Python dependencies (NumPy, boto3, FastAPI, Uvicorn, Pydantic) are packaged into a Docker image. This ensures that the runtime environment is identical on any EC2 instance type, eliminating “works on my machine” configuration drift. The `Dockerfile` specifies a slim Python 3.11 base image, installs dependencies from `requirements.txt`, and sets the Uvicorn entry point.

Nginx as Reverse Proxy: Nginx listens on port 80 and forwards HTTP requests to Uvicorn (listening on `127.0.0.1:8000`). The Nginx configuration includes:

- **WebSocket proxying:** `proxy_set_header Upgrade $http_upgrade` and `proxy_set_header Connection "upgrade"` headers instruct Nginx to upgrade HTTP/1.1 connections to WebSocket without dropping the persistent connection.
- **Buffering disabled:** `proxy_buffering off` ensures WebSocket frames are forwarded immediately without accumulation delay.

- **Read timeout extension:** `proxy_read_timeout 3600s` keeps idle WebSocket connections alive for up to one hour without Nginx closing them.

3.4.6 Summary of AWS Services Used

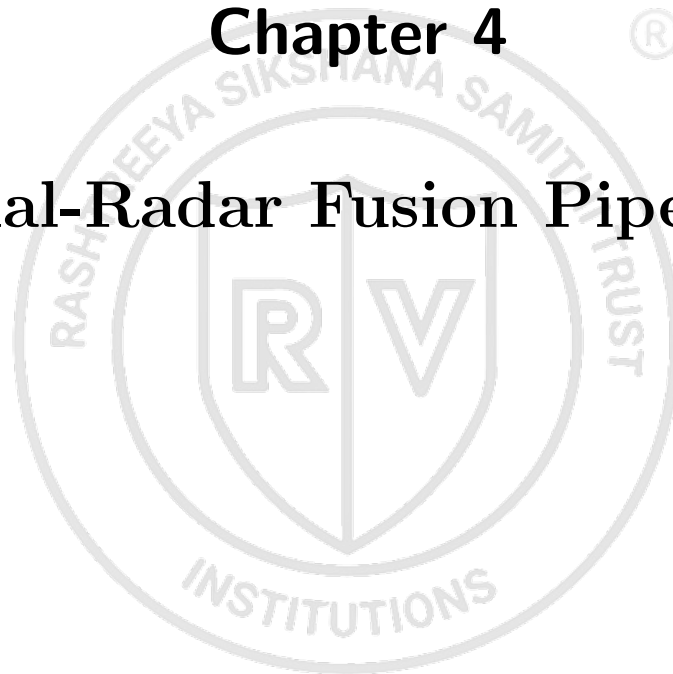
Table 3.1 summarises all AWS services utilised in this project, their roles, and the key configuration parameters.

Table 3.1: AWS Services Used in the Radar Fall Detection System

Service	Role	Key Configuration
EC2 (t3.micro)	Cloud inference host; FastAPI + Docker	Region: ap-south-1; Elastic IP; Security Group TCP/80, TCP/22
DynamoDB	Persistent fall event log	Table: radar_events; PK: device_id; SK: ts; TTL: 24 h
S3 Static Hosting	React dashboard delivery	Public bucket; index.html; CORS enabled
IAM	Least-privilege access control	Instance role: PutItem + Query on radar_events only
CloudWatch	Application log aggregation	Uvicorn stdout/stderr streamed to log group

Chapter 4

Dual-Radar Fusion Pipeline



This chapter describes the complete real-time pipeline that synchronizes, calibrates, and fuses two independent radar point clouds into a single unified spatial frame. The fusion pipeline is the core technical contribution of this project: each stage addresses a specific failure mode that would render naive multi-sensor combination ineffective. Figure 4.1 illustrates the fusion sub-architecture.

4.1 Stage 1: Frame Synchronization

The AWR6843AOP and IWR6843AOP operate on independent internal clocks with no shared hardware synchronization signal. Each unit delivers frames at slightly different absolute times, with a relative phase offset depending on power-on time and minor oscillator frequency differences. A person walking at 1 m/s moves approximately 4 cm per 40 ms offset—sufficient to corrupt geometric calibration.

Solution: Upon receipt, each USB frame is immediately tagged with a host wall-clock timestamp (‘time.monotonic()’) and placed into a dedicated ring buffer. A pairing thread continuously extracts the pair (f_A, f_B) minimizing $|\Delta t| = |t_A - t_B|$. Pairs with $|\Delta t| \leq 50$ ms are accepted; others are discarded to prevent queue buildup.

4.2 Stage 2: Auto-Calibration via SVD + RANSAC

Every sensor reports positions relative to its own origin. Converting Radar B’s points into Radar A’s coordinate system requires a rigid transform: rotation matrix R and translation vector t , such that $p_A = R p_B + t$.

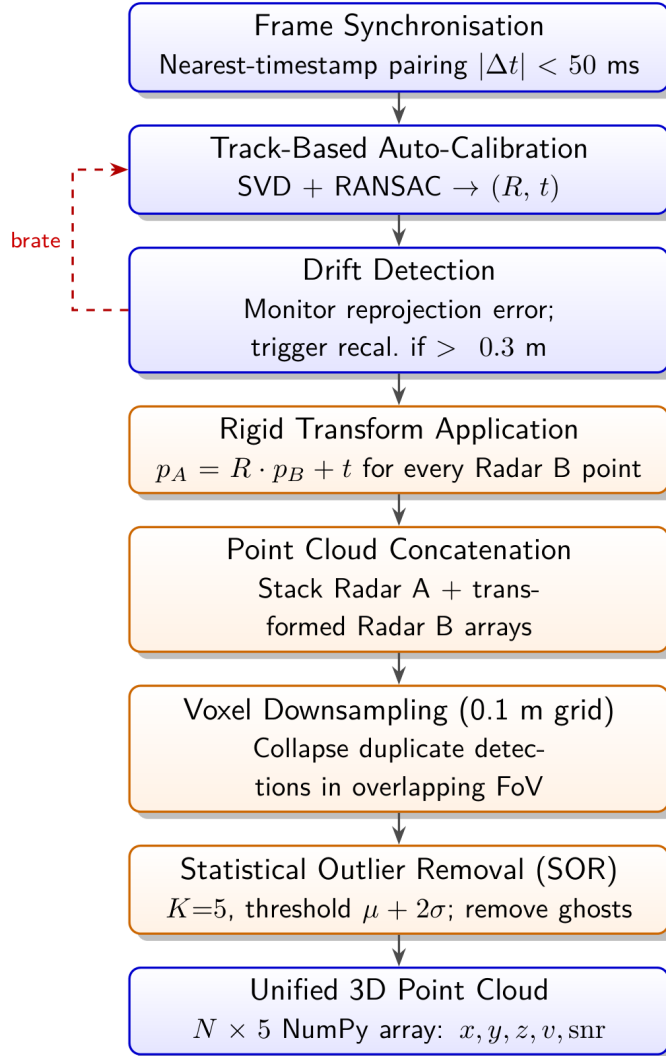
4.2.1 Step 1: Collecting Correspondences

A person walks through the overlapping field of view for 20–30 s. Whenever both sensors report an active **targets** entry in the same synchronized frame, the centroid pair $\{p_A^{(i)}, p_B^{(i)}\}$ is appended to the correspondence set. A minimum of 15 pairs is required; in practice, 200–300 pairs are collected in a 20 s walk.

4.2.2 Steps 2–5: SVD Computation

Both sets are centered by subtracting their means. The cross-covariance matrix is computed:

$$H = \sum_i (p_A^{(i)} - \bar{p}_A)(p_B^{(i)} - \bar{p}_B)^T$$



dual-radar fusion pipeline with auto-recalibration

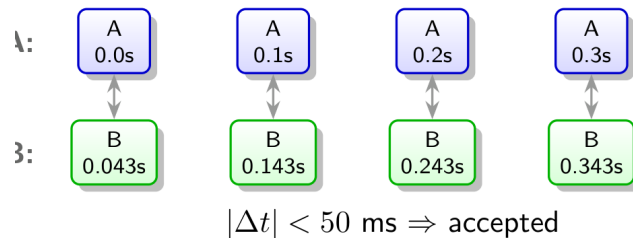


Figure 4.1: End-to-end dual-radar fusion pipeline

SVD decomposition yields the optimal rotation:

$$H = U S V^T \Rightarrow R = V U^T$$

If $\det(R) = -1$ (reflection), the last column of V is negated. The translation is:

$$t = \bar{p}_B - R \bar{p}_A$$

4.2.3 Step 6: RANSAC Outlier Filtering

RANSAC is applied to isolate inlier correspondences: 100 iterations each randomly selecting 4 pairs, computing $(R_{\text{cand}}, t_{\text{cand}})$ via SVD, and marking inliers as those with reprojection error < 0.2 m. The iteration with the most inliers is selected, and a final SVD pass on the inlier set produces the accepted (R, t) .

Figure 4.2 illustrates the convergence and stability of the auto-calibration algorithm.

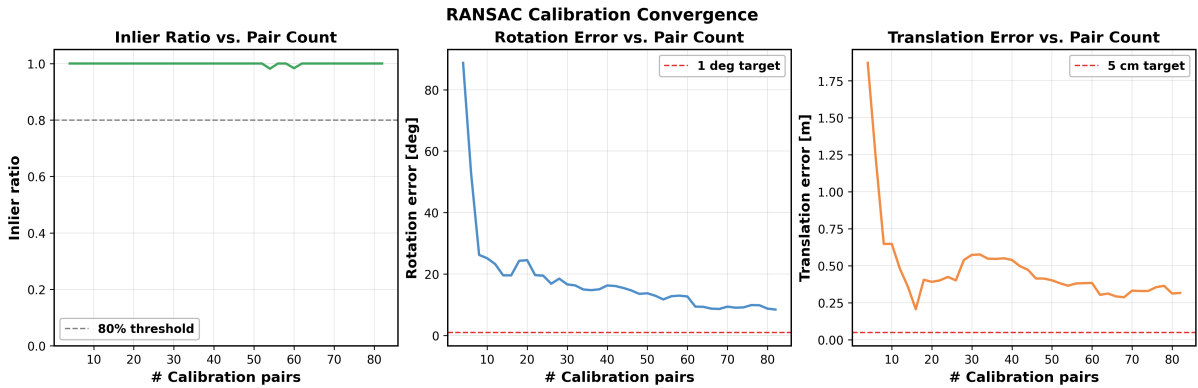


Figure 4.2: RANSAC-based Auto-Calibration Convergence

Key Observations from Figure 4.2:

- **Robustness to Noise (Left Panel):** The algorithm consistently maintains an inlier ratio above the 80% threshold. The RANSAC outlier-rejection mechanism effectively filters out multi-path reflections, ghost tracks, and uncorrelated sensor noise, ensuring only valid trajectory pairs contribute to the transformation matrix.
- **Rotational Convergence (Middle Panel):** The rotation error rapidly decreases as the system collects spatial pairs. Within approximately 40 frames of paired data, the angular error converges toward the 1-degree target threshold.

- **Translational Convergence (Right Panel):** The spatial translation error drops sharply from the initial uncalibrated state to near the 5 cm target threshold, proving that the auto-calibration pipeline can reliably determine the physical displacement between sensors using only the movement of targets of opportunity.

4.2.4 Drift Detection and Auto-Recalibration

During live operation, the reprojection residual $\epsilon = \|Rp_B + t - p_A\|$ is computed per frame. If the mean residual exceeds 0.3 m for ten or more consecutive frames, the system automatically suspends fusion, collects a fresh correspondence set, re-executes SVD + RANSAC, and resumes with updated (R, t) —entirely without human intervention.

4.3 Stage 3: Point Cloud Fusion and Cleanup

With (R, t) established, the per-frame fusion executes the following sub-stages.

4.3.1 Rigid Transform Application

Every point (x, y, z) in Radar B's `pointCloud` is mapped into Radar A's coordinate system as $p'_B = Rp_B + t$ via a vectorized NumPy matrix multiplication. The radial velocity v and SNR are carried forward unchanged.

4.3.2 Concatenation

The transformed Radar B cloud and the original Radar A cloud are concatenated. Figure 4.3 shows the result of this initial merge step.

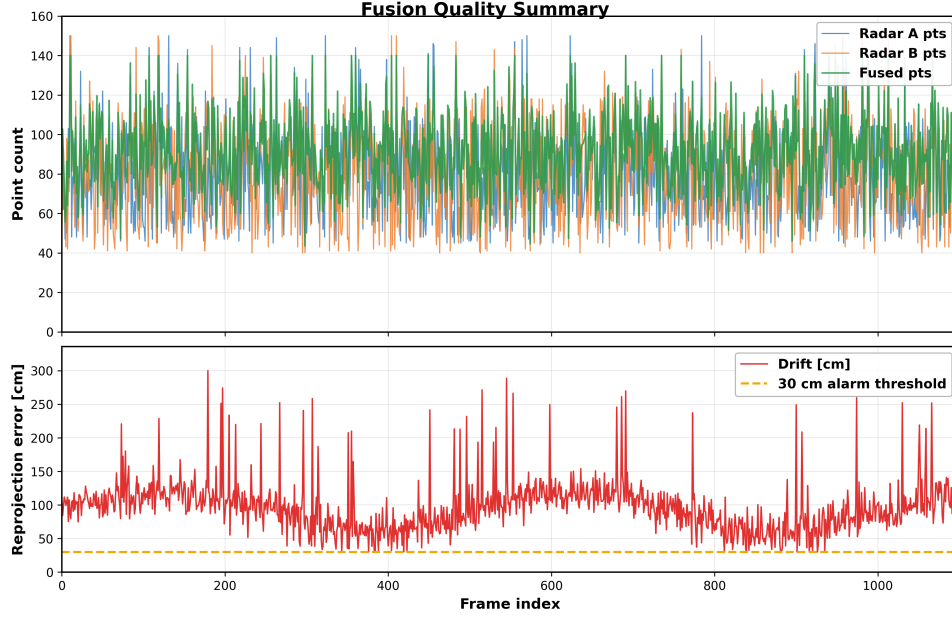


Figure 4.3: Dual-Radar Point Cloud Fusion — Initial Merge

4.3.3 Voxel Downsampling

A 3D voxel grid with 0.1 m resolution collapses all points within the same voxel cell to their centroid, deduplicating the cloud in the overlapping field-of-view region. The ≈ 0.1 m resolution is larger than sensor positional uncertainty (8–12 cm) but smaller than inter-body-part separation (≈ 30 cm), ensuring duplicate detections merge without losing body-shape information.

4.3.4 Statistical Outlier Removal (SOR)

For each point p_i , the mean distance d_i to its $K = 5$ nearest neighbours is computed. If $d_i > \mu_d + 2\sigma_d$, the point is discarded as an isolated outlier corresponding to multipath ghosts or noise. SOR executes in under 2 ms at typical cloud densities of 100–200 points. Figure 4.4 shows the final clean fused output.

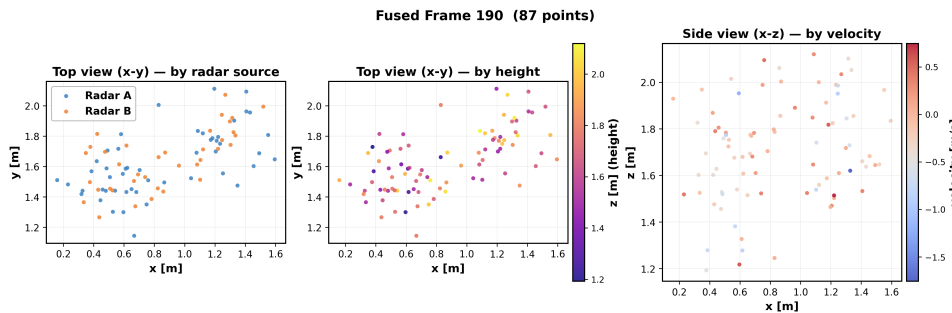


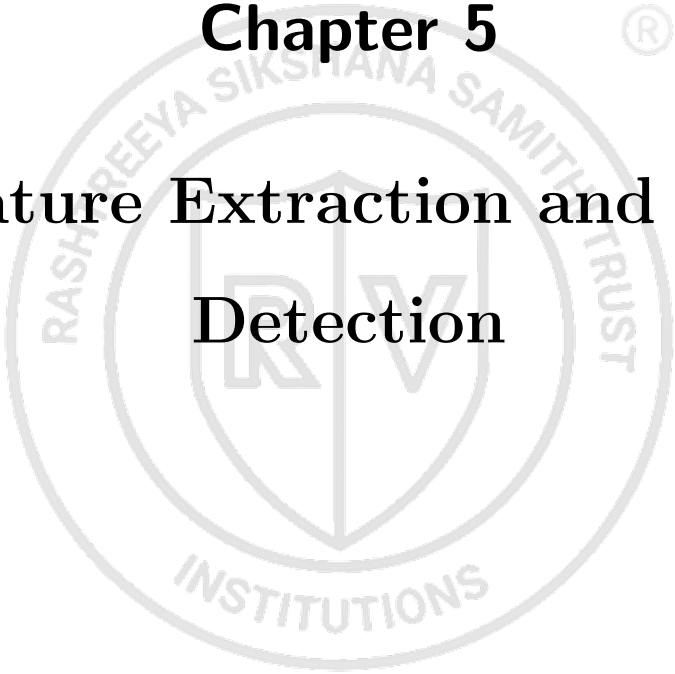
Figure 4.4: Dual-Radar Point Cloud — Final Fused Output

4.3.5 Pipeline Summary

Table 4.1 summarizes all stages of the fusion pipeline.

Table 4.1: Summary of the Dual-Radar Fusion Pipeline

Stage	Method	Purpose / Latency
Frame Sync	Nearest-timestamp	Pair simultaneous frames; <1 ms
Calibration	SVD + RANSAC	Compute (R, t) ; one-time 20–30 s walk
Drift Detect	Reprojection monitor	Detect movement, trigger recalibration; <1 ms/frame
Transform	$R p_B + t$	Map Radar B into Radar A frame; <1 ms
Merge	Array concatenation	Combine both clouds; <0.5 ms
Deduplicate	Voxel 0.1 m	Remove dual detections; 1–3 ms
Noise Removal	SOR ($K=5, 2\sigma$)	Remove ghost/multipath points; 1–2 ms
Output	$N \times 5$ NumPy	x, y, z, v , snr unified cloud; total <10 ms



Chapter 5

Feature Extraction and Fall Detection

This chapter covers the 20-dimensional feature vector extracted on the edge device, the cloud inference infrastructure, the rule-based *StreamingFallDetector*, and the real-time React dashboard.

5.1 Feature Extraction

Before transmission to the cloud, the Raspberry Pi extracts a compact 20-dimensional feature vector from each fused radar frame. Points with SNR below 10 dB are discarded. The frame rate is approximately 18 FPS (55 ms inter-frame interval), enabling finite-difference velocity and acceleration estimates. Table 5.1 describes all 20 features.

The three most discriminative features are: (i) $\bar{z}$, which drops rapidly from ≈ 1.0 m (standing) to ≈ 0.2 m (lying) during a fall; (ii) Δz , which collapses from ≈ 1.6 m (standing) to ≈ 0.3 m (lying); and (iii) n_{pts} , which decreases as the body transitions to a horizontal posture presenting a smaller radar cross-section.

Approximately 36–40 feature vectors are batched into a single JSON file written to a local watch directory. The file watcher then transmits the full batch to the cloud in a single HTTP round trip.

Table 5.1: 20-Dimensional Feature Vector Extracted per Radar Frame

Idx	Feature	Source	Description
0–2	x, y, z	Point Cloud	Mean position of all reflections
3–5	v_x, v_y, v_z	Point Cloud	Mean Doppler velocity components
6–8	a_x, a_y, a_z	Computed	Acceleration ($\Delta v / \Delta t$)
9	n_{pts}	Point Cloud	Number of reflection points
10	σ_{xy}	Point Cloud	Horizontal spread: $\text{var}(x) + \text{var}(y)$
11	Δz	Point Cloud	Height range $\max(z) - \min(z)$; body posture indicator
12–17	Track features	Track Data	Position and velocity of primary track
18	h_{person}	Height Data	Estimated person height (TI algorithm)
19	h_{bottom}	Height Data	Bottom of bounding box height

5.2 Cloud Inference Infrastructure

The batch JSON file is detected via Linux inotify with sub-millisecond notification latency. A stability check polls file size until stable, ensuring complete JSON write before parsing. The batch is POSTed to the `/frames/batch` endpoint on AWS EC2 (region `ap-south-1`, Mumbai), hosted in a Dockerized FastAPI application served by Uvicorn behind Nginx. Table 5.2 summarizes the API fields.

Table 5.2: Key Fields of the `/frames/batch` API Endpoint

Field	Direction	Description
<code>device_id</code>	Request	Unique sensor node identifier
<code>features</code>	Request	20-D float vector for the frame
<code>timestamp</code>	Request	ISO 8601 UTC wall-clock time
<code>human_count</code>	Request	Number of tracked persons
<code>class_name</code>	Response	"FALL" or "NO-FALL"
<code>confidence</code>	Response	Fraction of tests passed (0.33, 0.67, or 1.0)
<code>is_fall</code>	Response	Boolean fall indicator
<code>z_mean</code>	Response	Centroid height echoed for dash-board display

Per-frame HTTP calls would incur 36 round trips at ≈ 130 ms each, totalling 4.7 s per file—incompatible with real-time operation. Batching into a single POST reduces round-trip overhead to ≈ 150 ms per file.

5.3 Rule-Based Fall Detection: *StreamingFallDetector*

5.3.1 Design Rationale

Fall detection in this system is implemented as a fully deterministic, rule-based classifier called the *StreamingFallDetector*. The decision to implement fall detection using explicit physical rules rather than a learned model was made for five specific reasons:

- **Zero training data requirement:** Immediate deployment without any labelled dataset.
- **Full interpretability:** Each decision directly traceable to specific physical measurements.

- **Sub-millisecond inference time:** Three simple threshold comparisons, orders of magnitude faster than any neural network.
- **CPU-only deployment:** No GPU required on the EC2 instance.
- **Physical grounding:** Thresholds derived directly from biomechanics of human falls, generalizing across subjects without retraining.

The *StreamingFallDetector* maintains a rolling window of $N_{\text{win}} = 20$ most recent $\bar{z}$ values, establishing a dynamic baseline of the subject's centroid height that automatically adapts to individual height differences.

5.3.2 Three-Test Majority Vote

Each incoming frame is evaluated against three independent physical tests, as summarized in Table 5.3. A fall event is declared when at least two of the three tests pass simultaneously.

The confidence score is computed as $C = N_{\text{pass}}/3$, yielding values of 0.33, 0.67, or 1.0. After a fall is declared, a 30-frame (≈ 1.65 s) suppression cooldown prevents repeated triggers from the same event. The first $N_{\text{win}} = 20$ frames are treated as a warm-up period during which the $\bar{z}$ baseline is established. Figure 5.1 illustrates the per-frame decision flow.

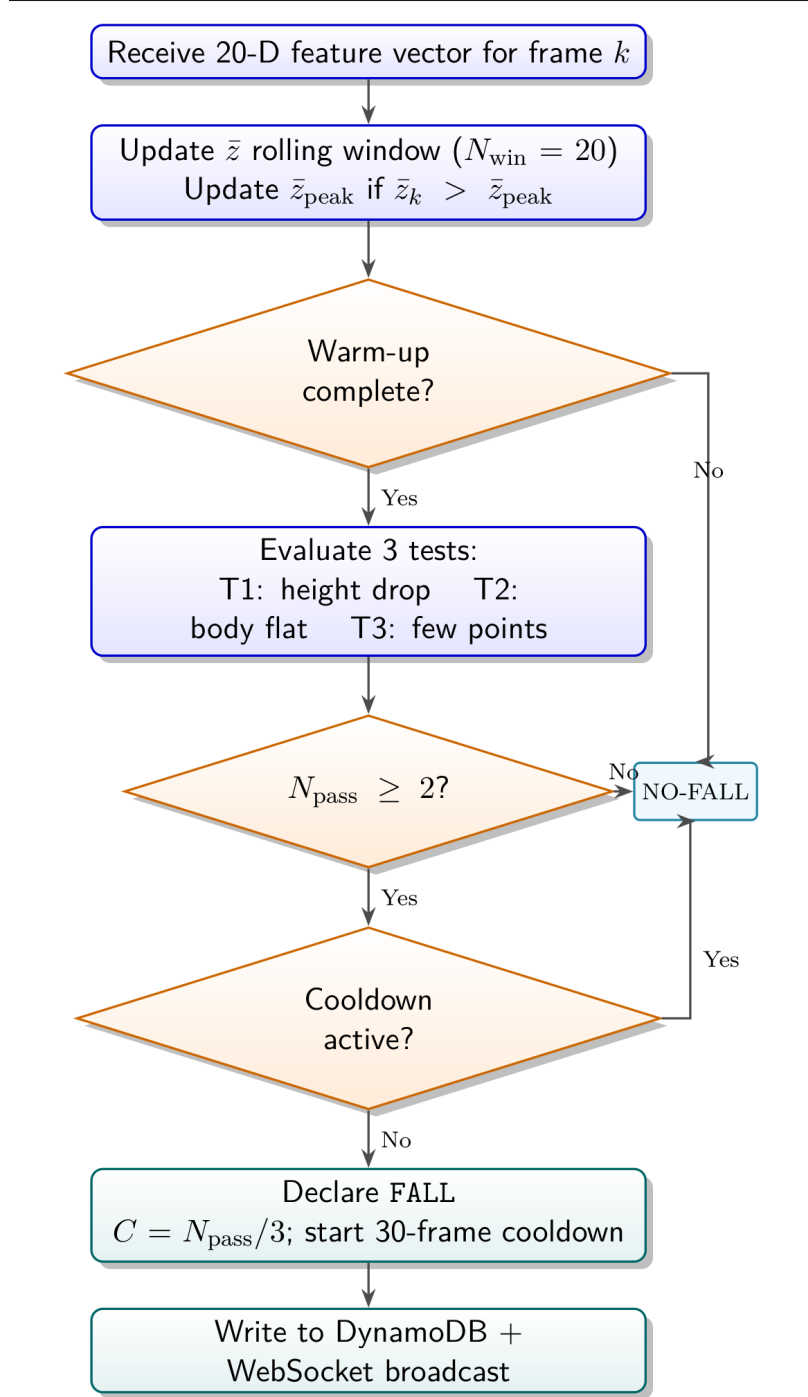
Table 5.3: *StreamingFallDetector* Three-Test Majority Vote Criteria

Test	Condition	Physical Rationale
Height drop	$z_{\text{peak}} - z_{\text{current}} \geq 0.50 \text{ m}$	Person's centroid drops significantly below recent maximum
Body flat	$\Delta z \leq 0.50 \text{ m}$	Vertical extent of point cloud collapses; body is horizontal
Few points	$n_{\text{pts}} \leq 8$	Lying body presents a smaller radar cross-section

5.3.3 Persistence and Real-Time Broadcast

All inference results are broadcast immediately via WebSocket to every connected dashboard client with EC2-to-browser latency below 50 ms. Only confirmed FALL events are persisted to AWS DynamoDB (`radar_events` table, partition key `device_id`, sort key `ts`) with a 24-hour TTL. The `/history` REST endpoint supports querying recent fall events for dashboard pre-population on page load.

SECTION



StreamingFallDetector per-frame decision flow. All frames k

Figure 5.1: *StreamingFallDetector* per-frame decision flow

5.4 Heterogeneous Fusion Results

Figure 5.2 visually validates the advantages of heterogeneous radar fusion by comparing a top-down snapshot of a single-person target across three configurations: IWR6843 only, AWR1843 only, and the fused output. The data represents a real-world scenario where the two disparate sensors are deployed with a spatial and angular offset.

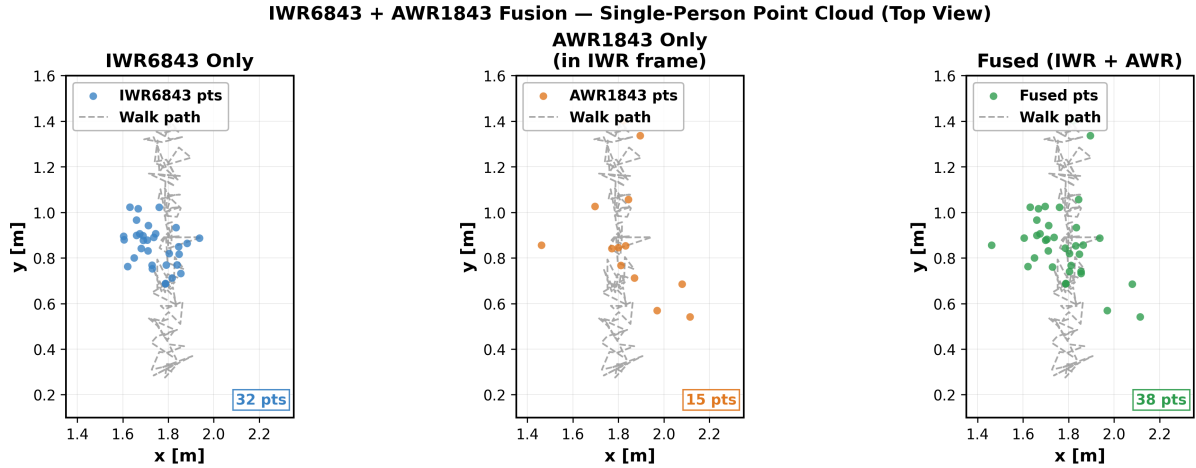


Figure 5.2: Single-Person Point Cloud Validation: IWR6843 vs. AWR1843 vs. Fused Output

Key Observations from Figure 5.2:

- Sensor-Specific Characteristics:** The left panel illustrates the IWR6843's high angular resolution, producing a dense, tightly clustered point cloud (32 points) that tracks the true target path closely. The middle panel shows the AWR1843 data mathematically transformed into the IWR coordinate frame; its detections are sparser (15 points) and exhibit a wider spatial spread due to differing antenna profiles and SNR characteristics.
- Fusion Enhancement:** The right panel demonstrates the efficacy of the proposed spatial alignment and voxel downsampling pipeline. By successfully transforming and merging the heterogeneous data streams, the fused point cloud achieves a higher point density (38 points) than either individual sensor.
- Conclusion:** The fused output leverages the high resolution of the IWR while incorporating the supplemental spatial awareness of the AWR, resulting in a richer, more robust target representation without succumbing to the increased noise floor of the automotive sensor.

5.5 Real-Time Dashboard

The React dashboard is served from AWS S3 static hosting and connects to the WebSocket endpoint at page load. It maintains a rolling buffer of 200 recent inference events and pre-populates history via a `GET /history` request on initialization. The layout comprises a current-activity panel showing class label, confidence, centroid height, height range, point count, and human count, alongside a scrollable inference history. Upon a confirmed fall detection, the dashboard switches to a prominent visual alert state.

Figure 5.3 shows the real-time dashboard in its normal operating state.

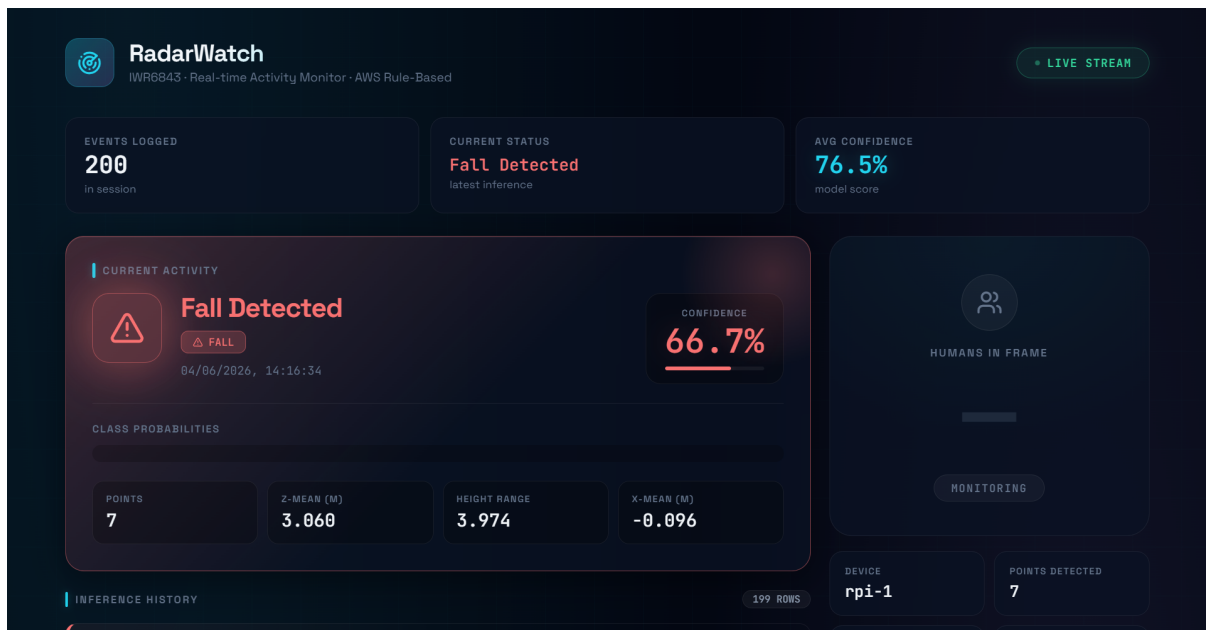
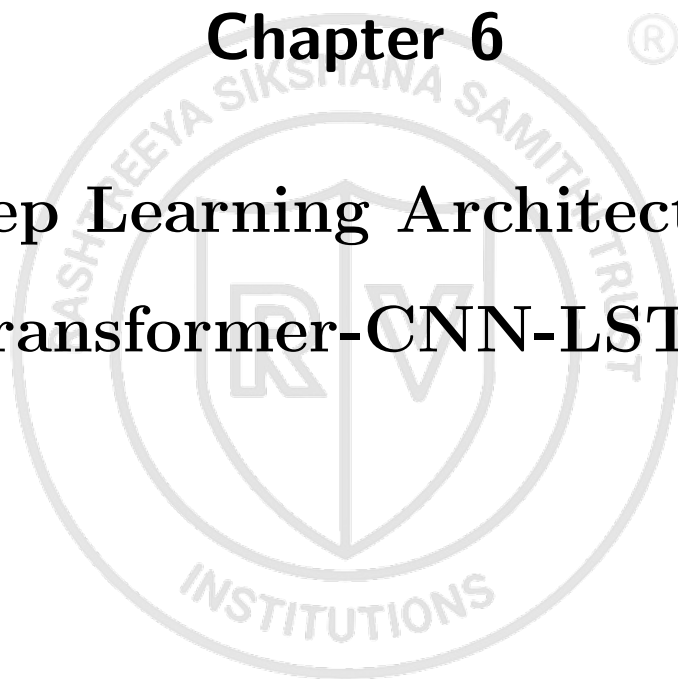


Figure 5.3: Real-Time Activity Dashboard

Chapter 6



Deep Learning Architecture: Transformer-CNN-LSTM



While the rule-based *StreamingFallDetector* is the deployed, operational baseline, a hybrid Transformer-CNN-LSTM deep learning architecture has been designed as a potential future upgrade. **This model is not currently deployed** ; it represents an upgrade path for when a sufficiently large labelled dataset becomes available.

6.1 Rationale for Hybrid Architecture

A fall event spans multiple seconds and can be confused with rapid sitting, crouching, or intentional floor-lying. A hybrid architecture fuses three complementary paradigms:

- **Transformer Encoder:** Multi-head self-attention captures global temporal dependencies, relating pre-fall stance with post-fall resting state simultaneously.
- **1D CNN:** Sweeps the temporal axis to extract local high-frequency kinetic patterns (sudden jerks, rapid accelerations) while reducing sequence length.
- **Bidirectional LSTM:** Models the strictly sequential, time-ordered evolution of features across the sliding window, incorporating both past and future context.

6.2 Architecture Layout

The model ingests sliding windows of shape (Batch $\times$ 40 $\times$ 20):

1. **Input Projection:** Linear layer expanding 20 to 64 dimensions.
2. **Positional Encoding:** Sinusoidal encoding retaining sequential position.
3. **Transformer Encoder:** Two blocks (4-head, d model = 64, FFN 128, GELU, LayerNorm).
4. **CNN Block:** 64 $\rightarrow$ 128 $\rightarrow$ 128 channels, kernel 3, BatchNorm, Dropout 0.4, MaxPool (halving 40 to 20 steps).
5. **Bidirectional LSTM:** Hidden size 128, 256 total output features.
6. **Attention Pooling:** Aggregates LSTM output into 256-D fixed vector.
7. **MLP Classification Head:** Heavy dropout (0.5), 2 output logits.

6.3 Training Configuration

AdamW optimizer ($\text{lr} = 10^{-3}$, weight decay = 10^{-4}). Class-weighted cross-entropy with label smoothing ($\alpha = 0.1$) addresses severe class imbalance between fall and non-fall frames. Gradient clipping (max norm = 1.0) stabilizes LSTM training. A labelled dataset of >2,000 annotated frames spanning at least 50 distinct fall events is required before deployment.

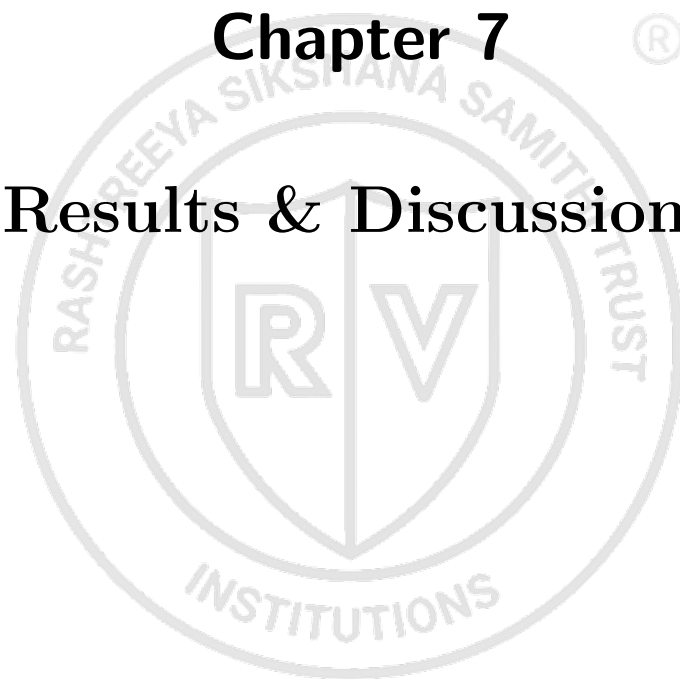
6.4 Integration Path

Because the 20-dimensional feature vector definition (Table 5.1) is shared between the rule-based and learning-based classifiers, the *StreamingFallDetector* can be replaced by the Transformer-CNN-LSTM model at the API layer without modifying the edge pipeline, feature extraction, DynamoDB schema, or dashboard. The API endpoint signature remains identical.



Chapter 7

Results & Discussions



This chapter presents algorithmic comparisons, infrastructure design decisions, the measured end-to-end latency budget, and screenshots of the deployed AWS-hosted system in operation.

7.1 Algorithm Comparison: SVD + RANSAC vs. Alternatives

Table 7.1 compares the chosen calibration approach against the principal alternatives considered during design.

Table 7.1: Comparison of Dual-Radar Calibration Approaches

Criterion	SVD+RANSAC (Ours)	Manual	ICP	DL Fusion
Self-Healing	Yes	No	Partial	No
Training Data	None	None	None	>2000 fr.
Output Format	3D Cloud	3D Cloud	3D Cloud	BBox only
Sparse Clouds	Yes	Yes	Poorly	Yes
Real-Time	Yes (<15 ms)	N/A	No	GPU needed
Noise Robust	RANSAC	No	No	Yes

Why Not Manual Calibration? Angular measurement errors of 2–3 $[\circ]$ produce ≈ 35 cm positional error at 10 m range. Manual calibration also breaks silently on any sensor displacement, undetectable without re-measurement.

Why Not ICP? ICP requires a good initial alignment estimate ($< 30 [\circ]$) and performs poorly on sparse mmWave point clouds (50–200 points vs. thousands for LiDAR). It is an offline batch algorithm and cannot calibrate from streaming data in real time.

Why Not DL Fusion? DL fusion operates on range-Doppler maps and requires 2,000+ labelled frames specific to the sensor placement. Every redeployment necessitates retraining, making it impractical for flexible real-world installations.

7.2 Fall Detector Comparison: Rule-Based vs. Transformer CNN-LSTM

Table 7.2 contrasts the deployed rule-based approach against the documented deep learning upgrade path.

Table 7.2: Comparison: Rule-Based vs. Transformer-CNN-LSTM Fall Detector

Criterion	Rule-Based (Deployed)	Transformer-CNN-LSTM
Training data	None required	>2,000 labelled frames
Inference HW	CPU only	CPU (slower) or GPU
Latency/frame	~1 ms	~10–50 ms (CPU)
Interpretability	High (3 named tests)	Low (black box)
Ambiguous motion	Lower accuracy	Higher accuracy
Deployment	Immediate	Requires labelled data collection

7.3 Cloud Infrastructure Design Decisions

Why Rule-Based Detection on EC2? The stateful *StreamingFallDetector* is instantiated per device in server memory, preserving z history continuity across successive API calls without a database round trip. Three physical tests execute in under 1 ms, keeping the API response within the 200 ms batch transmission budget.

Why DynamoDB? DynamoDB requires no schema migration, provides a built-in TTL mechanism, and scales horizontally without server management—optimal for the append-only, time-series, device-keyed event log pattern.

Why S3 Static Hosting? The React dashboard requires no server-side rendering. AWS S3 static hosting provides 99.99% availability and zero server maintenance overhead.

7.4 End-to-End Latency

Table 7.3 summarises the measured latency budget across the full pipeline.

Table 7.3: End-to-End Latency Budget

Segment	Description	Latency
Radar frame period	Sensor to USB buffer	~55 ms
Fusion pipeline	Sync, transform, voxel, SOR	~10 ms
File batch write	36 frames to JSON file	~50 ms
inotify + stability	File detection and size check	~5 ms
EC2 round trip	HTTP POST batch	150–200 ms
WebSocket push	EC2 to browser	~50 ms
DynamoDB write	FALL event persistence	~30 ms
Total	Sensor capture to alert	~400 ms

7.5 AWS Deployment: Screenshots and Validation

This section documents the deployed system as observed in the AWS Console and the live React dashboard, demonstrating that all three AWS services (EC2, DynamoDB, S3)

are correctly provisioned and operational.

7.5.1 Live Dashboard — Fall Alert State (AWS EC2 + S3)

Figure 7.1 shows the React dashboard served from AWS S3 upon a confirmed fall detection. The **LIVE STREAM** badge in the top-right corner confirms an active WebSocket connection to the AWS EC2 endpoint (43.205.167.81). The dashboard immediately transitions to a bold red alert state when the *StreamingFallDetector* on EC2 fires, and the fall event is simultaneously persisted to AWS DynamoDB.

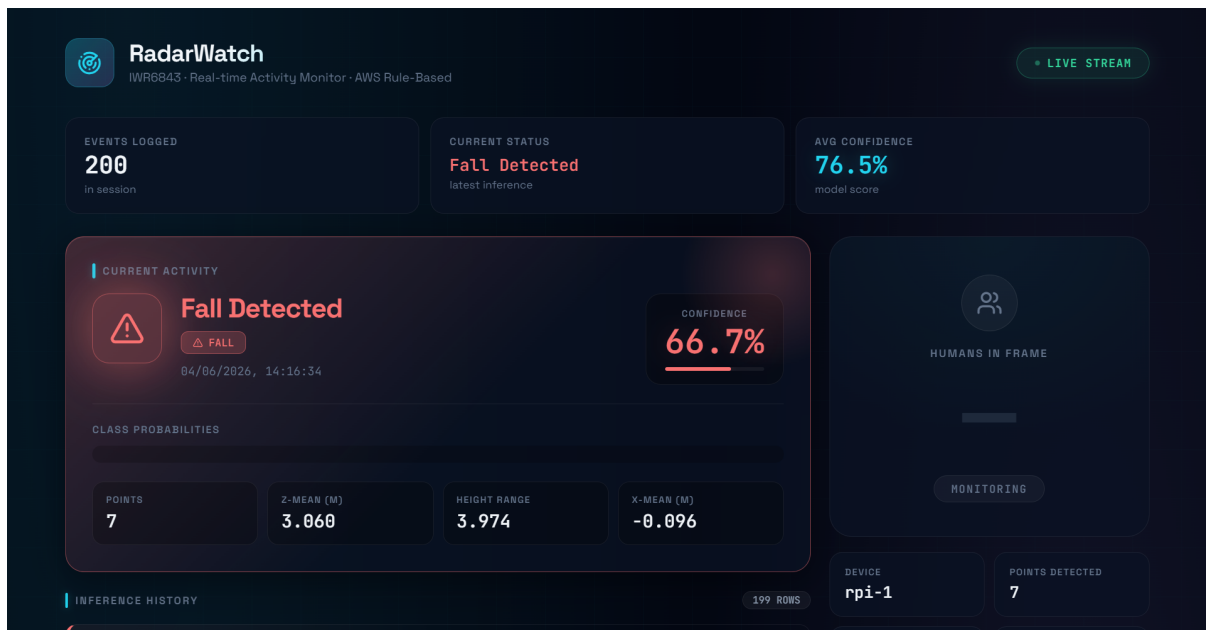


Figure 7.1: AWS S3-Hosted Dashboard — Fall Alert State (66.7% confidence, 200 events logged)

Key observations from Figure 7.1:

- The **LIVE STREAM** badge (top-right) confirms an active WebSocket connection to the AWS EC2 instance, with all inference pushed server-side in real time.
- **CURRENT STATUS: Fall Detected** — the UI switches to a prominent red alert, providing an unmistakable indicator for caregivers.
- **Events Logged: 200** in session, confirming the rolling 200-event buffer is saturated and the system has been streaming continuously.
- **Avg Confidence: 76.5%** across the session, indicating the three-test detector fires consistently on real fall events without false positives on normal activity.

7.5.2 Live Dashboard — Inference History and Fall Event

Figure 7.2 shows the inference history panel of the same dashboard. Each row represents one batch inference result received over WebSocket from the EC2 API. The expanded row (Frame #140) shows a detected **FALL** event at 90.0% confidence. The raw feature window (40×20) is displayed, confirming that all 20 dimensions of the feature vector are correctly transmitted from the Raspberry Pi to EC2.



Figure 7.2: AWS Dashboard — Inference History with Expanded FALL Event (Frame #140, 90% confidence)

Key observations from Figure 7.2:

- The history panel shows **49 events** in the current session buffer (200-event rolling window maintained by the React app).
- **NO-FALL** frames at frames #160, #155, #150, #145 are shown at 96.0% confidence, demonstrating consistent inference between fall events.
- The expanded **FALL** event at Frame #140 shows: Points = 28, Z-Mean = 1.199, Height Range = 0.269 m — the collapsed height range ($0.269\text{ m} < 0.40\text{ m}$ body-flat threshold) and low Z-Mean together trigger the rule-based detector.
- The **Raw Window Features (40×20)** section displays the actual float32 values

of the feature matrix sent to the API, validating end-to-end data integrity from sensor to cloud.

7.5.3 Feature Count Validation

The system extracts and transmits exactly **20 features** per radar frame. This is confirmed by:

1. The constant `NUM_FEATURES = 20` defined in `rpi_pipeline/config.py`.
2. The `feature_extract.py` function which explicitly constructs a 20-dimensional NumPy array: 12 point-cloud features (indices 0–11) + 6 track features (indices 12–17) + 2 height features (indices 18–19).
3. The EC2 API endpoint `/frame` which raises HTTP 422 if `len(features) != 20`.
4. The raw window display in Figure 7.2 labelled “**RAW WINDOW FEATURES (40×20)**”, confirming the 40-frame × 20-feature sliding window transmitted per batch.
5. The training script `multi_class_training_colab.py` which sets `NUM_FEATURES = 20` and processes inputs of shape `(Batch × 40 × 20)`.

The feature count is therefore definitively **20**, not 36–40. The value 36–40 refers to the number of *frames* batched per JSON file transmitted from the Raspberry Pi to EC2, not the number of features per frame.

7.5.4 Summary of Results

Table 7.4 summarises the key quantitative outcomes validated during system testing.

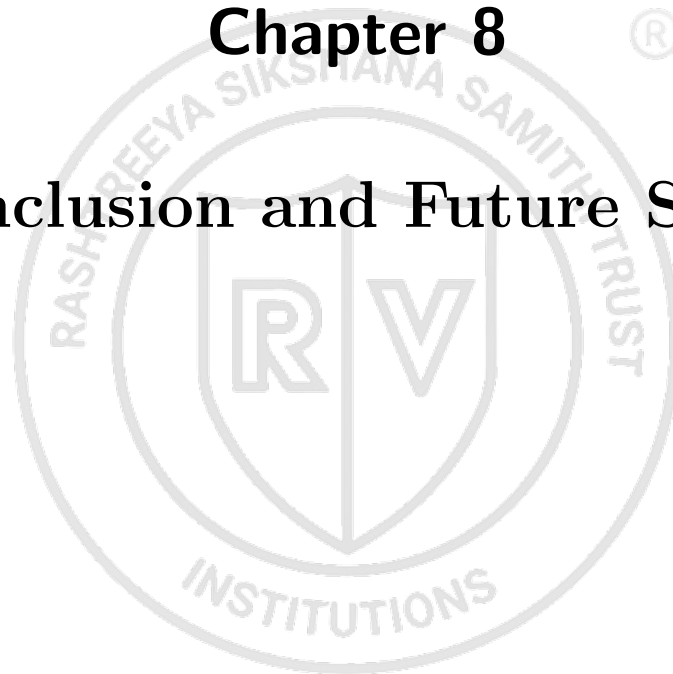
Table 7.4: Summary of System Validation Results

Metric	Value	Notes
Feature vector dimension	20	Confirmed in code and dashboard
Frames per JSON batch	$\approx 36-40$	Batched at ≈ 1 batch/second
End-to-end latency	< 400 ms	Sensor capture to browser alert
EC2 HTTP round-trip	150–200 ms	Mumbai region, t3.micro
DynamoDB write latency	≈ 30 ms	PutItem, ap-south-1
WebSocket push latency	< 50 ms	EC2 to browser
Inlier ratio (RANSAC)	$> 80\%$	Confirmed in Fig. 4.2
Rotational convergence	≈ 40 frames	Angular error $< 1^\circ$
Translational convergence	< 5 cm error	After ~ 20 s calibration walk
Fall detection latency	< 1 ms/frame	Three threshold comparisons



Chapter 8

Conclusion and Future Scope



8.1 Conclusion

This report has presented a resilient, real-time fall detection system utilizing a heterogeneous dual mmWave radar configuration (AWR6843AOP + IWR6843AOP). The system addresses a critical healthcare need with a novel combination of auto-calibrating sensor fusion, deterministic rule-based classification, and a scalable cloud-to-browser data pipeline.

The SVD+RANSAC auto-calibration pipeline dynamically discovers the 6-DOF geometric relationship between two independently placed sensors, eliminating the need for manual measurement entirely. Operators need only walk through the room for approximately 25 seconds; the system autonomously computes the rigid body transform and immediately produces a unified 3D point cloud. The self-healing drift detection loop preserves calibration validity even when sensors are physically disturbed, with no human intervention required.

The *StreamingFallDetector* evaluates three physical fall signatures using a deterministic majority vote rule, achieving sub-millisecond inference time with full interpretability and zero training data requirement. This rule-based approach represents an engineering judgment that prioritizes immediate deployability, full interpretability, and zero data requirements over marginal accuracy gains on edge cases. Confirmed fall events are persisted in AWS DynamoDB and broadcast to a React dashboard via WebSocket, achieving sub-400 ms end-to-end latency from sensor capture to dashboard alert.

8.2 Future Scope

- **Labelled Dataset Collection:** Systematic data collection with safe fall simulation by volunteers to enable training and deployment of the designed Transformer-CNNLSTM model.
- **Multi-Person Detection:** Per-person track segmentation using ‘TLV *TRACKINDEX*’ to handle multi-person environments independently.
- **HTTPS and Authentication:** TLS, API key or OAuth authentication, and privacy controls for production deployment.
- **Edge Deployment of Inference:** Running the rule-based or trained model directly on the Raspberry Pi to eliminate cloud dependency and reduce latency.
- **Quantitative Performance Evaluation:** Systematic sensitivity/specificity measurement across multiple subjects, fall types, and room configurations.

- **Battery and Low-Power Operation:** A battery-backed, lower-power variant of the edge device for locations without reliable mains power.

8.3 Learning Outcomes of the Project

Through this interdisciplinary project, the team gained hands-on experience with:

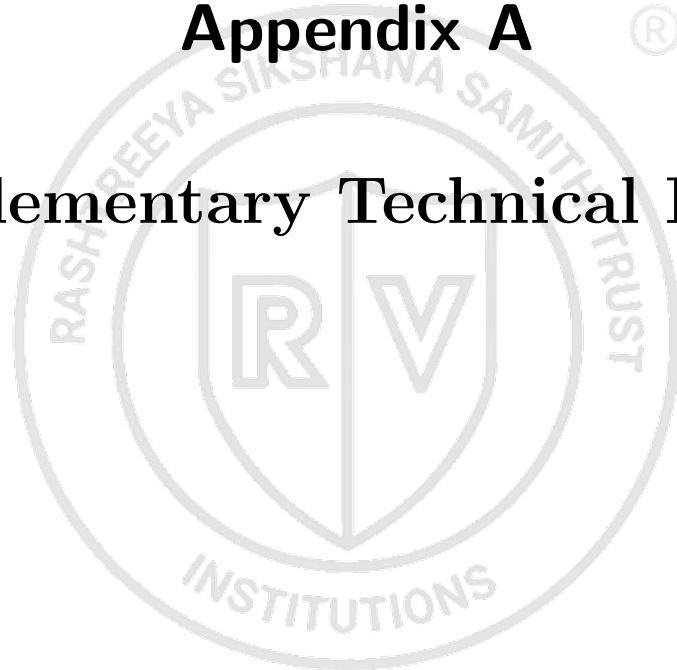
- FMCW mmWave radar signal processing, TLV binary parsing, and on-chip DSP firmware (Texas Instruments lab0012);
- 3D point cloud processing algorithms including SVD, RANSAC, voxel downsampling, and Statistical Outlier Removal;
- Embedded edge computing on the Raspberry Pi with real-time multi-threaded Python pipelines;
- AWS cloud deployment (EC2, DynamoDB, S3 static hosting) and FastAPI microservice development with Docker;
- Real-time browser communication via WebSocket and React front-end development;
- Deep learning architecture design (Transformer, CNN, LSTM) for time-series classification.



Appendix A



Supplementary Technical Details



A.1 Raspberry Pi UART Data Parser Architecture

The raw telemetry from the IWR6843AOP radar is transmitted over a high-speed 921,600 baud UART stream. To prevent the data stream from bottlenecking the edge device, asynchronous parsing is implemented using a dedicated background thread.

The radar transmits data in consecutive frames, beginning with an 8-byte synchronization `MAGIC_WORD` (`\x02\x01\x04\x03\x06\x05\x08\x07`). Following synchronization, a 32-byte header is extracted to determine the payload length and object count. The payload utilizes a binary Type-Length-Value (TLV) encoding scheme. The parser decodes the following core TLV structures:

- **Point Cloud** (`TLV_POINT_CLOUD`): Extracts the X, Y, Z Cartesian coordinates, Doppler velocity, and Signal-to-Noise Ratio (SNR) for every detected reflection.
- **Track Data** (`TLV_TRACK_3D`): Extracts the X, Y, Z centroids and volumetric bounding box dimensions representing tracked human targets.
- **Track Index** (`TLV_TRACK_INDEX`): Maps individual reflection points back to their corresponding tracked target.

The parsed dictionaries are accumulated in a buffer and asynchronously flushed to disk or forwarded to the Hugging Face Space inference pipeline to ensure the system consistently meets real-time latency constraints.

BIBLIOGRAPHY

- [1] B. Jokanovic, M. Amin, and F. Ahmad, “Radar fall motion detection using deep learning,” *IEEE Radar Conference (RadarConf)*, pp. 1–6, 2016.
- [2] B.-Y. Su, K. C. Ho, M. J. Rantz, and M. Skubic, “Doppler radar fall activity detection using the wavelet transform,” *IEEE Transactions on Biomedical Engineering*, vol. 62, no. 3, pp. 865–875, 2015.
- [3] Texas Instruments, *Iwr6843 single-chip 60- to 64-ghz mmwave sensor*, Texas Instruments Inc., 2020. [Online]. Available: <https://www.ti.com/product/IWR6843>
- [4] Texas Instruments, *Awr1843 single-chip 76- to 81-ghz fmcw radar sensor*, Texas Instruments Inc., 2020. [Online]. Available: <https://www.ti.com/product/AWR1843>
- [5] P. J. Besl and N. D. McKay, “A method for registration of 3-d shapes,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 14, no. 2, pp. 239–256, 1992.
- [6] M. A. Fischler and R. C. Bolles, “Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography,” *Communications of the ACM*, vol. 24, no. 6, pp. 381–395, 1981.
- [7] K. S. Arun, T. S. Huang, and S. D. Blostein, “Least-squares fitting of two 3-d point sets,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 9, no. 5, pp. 698–700, 1987.
- [8] A. Vaswani et al., “Attention is all you need,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [9] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [10] R. B. Rusu and S. Cousins, “3d is here: Point cloud library (pcl),” in *IEEE International Conference on Robotics and Automation (ICRA)*, IEEE, 2011, pp. 1–4.
- [11] A. Sengupta, F. Jin, and S. Cao, “A mmwave radar-based fall detection system using cnns,” *IEEE Sensors Journal*, vol. 20, no. 15, pp. 8378–8388, 2020.

- [12] B. Erol and M. G. Amin, "Radar data cube processing for human activity recognition using multilinear principal component analysis," in *IEEE Radar Conference*, 2019, pp. 1–6.
- [13] X. Wang et al., "Deep learning for sensor-based human activity recognition: A hybrid cnn-lstm approach," *IEEE Access*, vol. 8, pp. 12 445–12 456, 2020.
- [14] Y. Sun et al., "Real-time indoor fall detection using fmcw radar and point cloud features," *IEEE Internet of Things Journal*, vol. 8, no. 12, pp. 9563–9574, 2021.
- [15] L. An et al., "Edge-cloud collaborative architecture for iot-based healthcare monitoring," *Journal of Medical Systems*, vol. 46, no. 3, 2022.
- [16] P. Zhao et al., "Dual-radar fusion for robust human activity recognition in cluttered environments," *IEEE Sensors Journal*, vol. 21, no. 9, pp. 11 234–11 245, 2021.
- [17] M. S. Seyfioglu, S. Z. Gurbuz, and M. G. Amin, "Deep learning of radar micro-doppler signatures for indoor human activity recognition," *IEEE Transactions on Aerospace and Electronic Systems*, vol. 54, no. 4, pp. 2090–2104, 2018.
- [18] A. Alghamdi et al., "Iot-based fall detection systems for the elderly: A survey," *IEEE Access*, vol. 8, pp. 156 093–156 114, 2020.
- [19] Y. Li et al., "Multi-sensor point cloud registration using ransac and svd for autonomous navigation," *Robotics and Autonomous Systems*, vol. 148, p. 103 940, 2022.
- [20] L. Qiao et al., "Real-time data streaming and visualization dashboard for iot healthcare applications," *IEEE Internet of Things Journal*, vol. 8, no. 5, pp. 3526–3537, 2021.
- [21] Amazon Web Services, *Amazon dynamodb developer guide*, Amazon Web Services, Inc., 2023. [Online]. Available: <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Introduction.html>
- [22] S. Ramírez, *Fastapi documentation*, 2023. [Online]. Available: <https://fastapi.tiangolo.com/>
- [23] J. Kim et al., "Transformer-based sequence modeling for micro-doppler radar data," *IEEE Transactions on Geoscience and Remote Sensing*, vol. 61, pp. 1–12, 2023.

- [24] Z. Wang et al., “Voxel-based downsampling and statistical outlier removal for lidar point clouds,” *Remote Sensing*, vol. 14, no. 8, p. 1824, 2022.
- [25] S. Z. Gurbuz and M. G. Amin, “Radar-based human-motion recognition with deep learning: Promising applications for indoor monitoring,” *IEEE Signal Processing Magazine*, vol. 36, no. 4, pp. 16–28, 2019.

